Combinational logic

Abhilash Jindal

Agenda

Build and analyse common combinational circuits

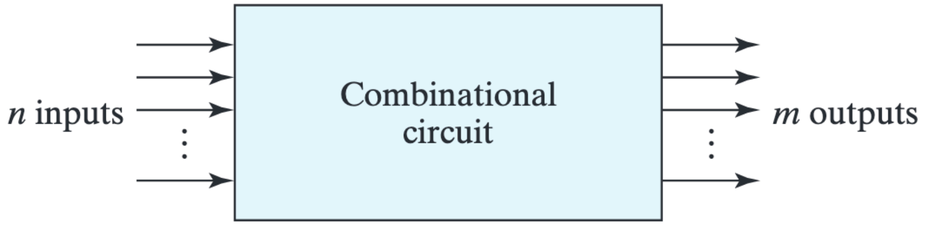
- Binary adders, subtractors, and multipliers: $x + y$, $x - y$, $x * y$
- Comparators. $x > y$
- Decoders: n to 2^n
  - Given an n-bit address, memory needs to find the byte from $\{0,\ldots,2^{n-1}\}$
  - Given an instruction opcode, CPU needs to decode it and activate the right circuit
- Encoders: 2^n to n
  - When a keyboard key is pressed, convert it to an ASCII code.
- Multiplexers: s ? y : z

Combinational circuits

- Combinational circuits: Output at any time is a function of inputs. Output is generated by *combining* inputs
  - Adder: add two input numbers to produce an output number
- Sequential circuits: Outputs are a function of the current inputs and the *state* of the storage elements. State is a function of *previous* inputs
  - Counter: Increment previous value
  - Need to be specified by a time *sequence* of inputs and internal states

Combinational circuits

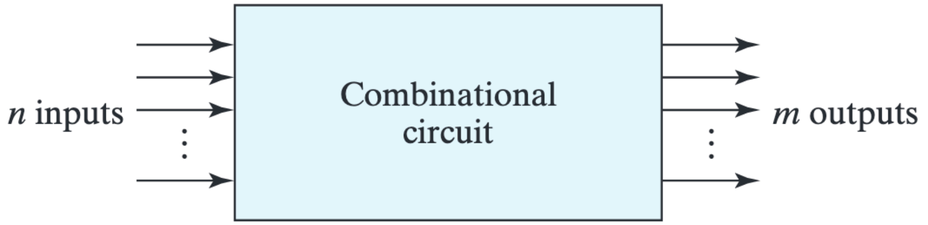
- n input variables, m output variables
  - Example: 4 input switches on Basys-3 board. 8 output cathodes for seven segment displays
- Input (output) could come from (go to) storage elements
  - Example: 4 input bits from a counter. 8 output cathodes for seven segment displays
  - Total circuit is a sequential circuit in this case



Combinational circuits

Possible circuits, Interconnection of logic gates

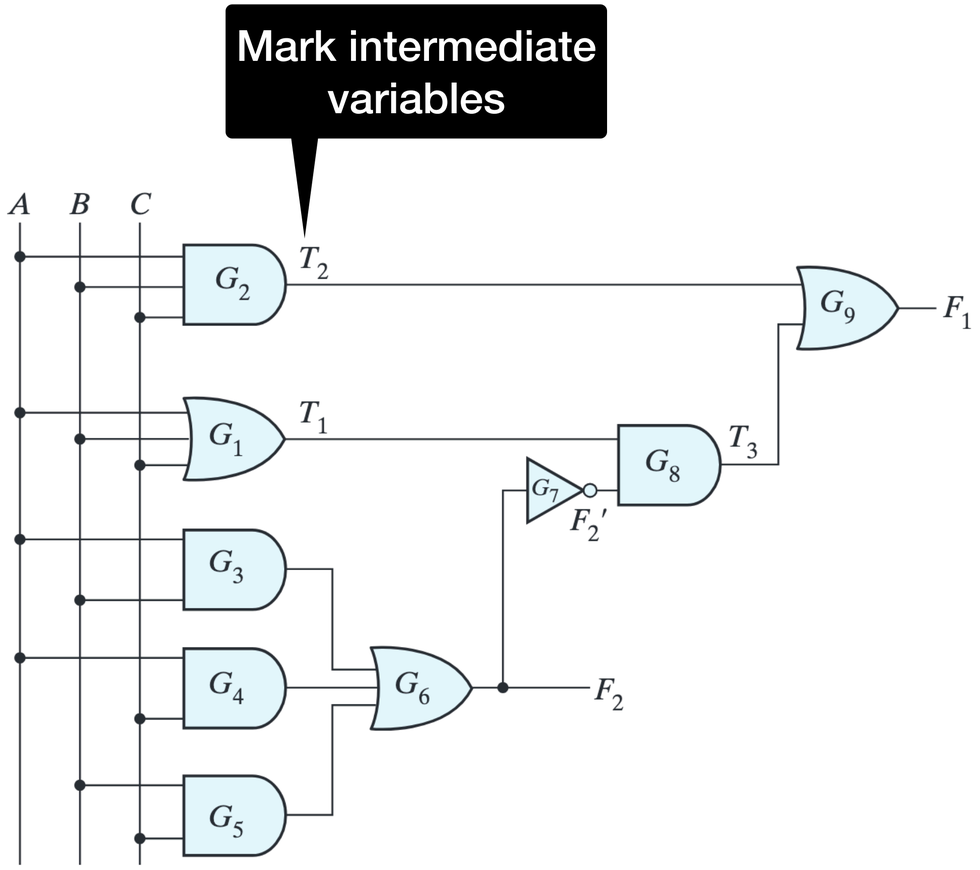
- n input variables, m output variables
  - 2^n possible input combinations
  - Each output is a Boolean function of n input variables: can be expressed with truth table, implemented with logic gates



Analysis of combinational circuits

What function does the circuit implement?

- $F_2 = AB + AC + BC$
- $T_1 = A + B + C$
- $T_2 = ABC$
- $T_3 = F'_2 T_1$
- $F_1 = T_3 + T_2$
  - $F_1 = F'_2 T_1 + ABC$
  - $F_1 = (AB + AC + BC)'(A + B + C) + ABC$
  - $F_1 = (A' + B')(A' + C')(B' + C')(A + B + C) + ABC$
  - $F_1 = (A' + B'C')(AB' + AC' + BC' + B'C) + ABC$
  - $F_1 = A'BC' + A'B'C + AB'C' + ABC$

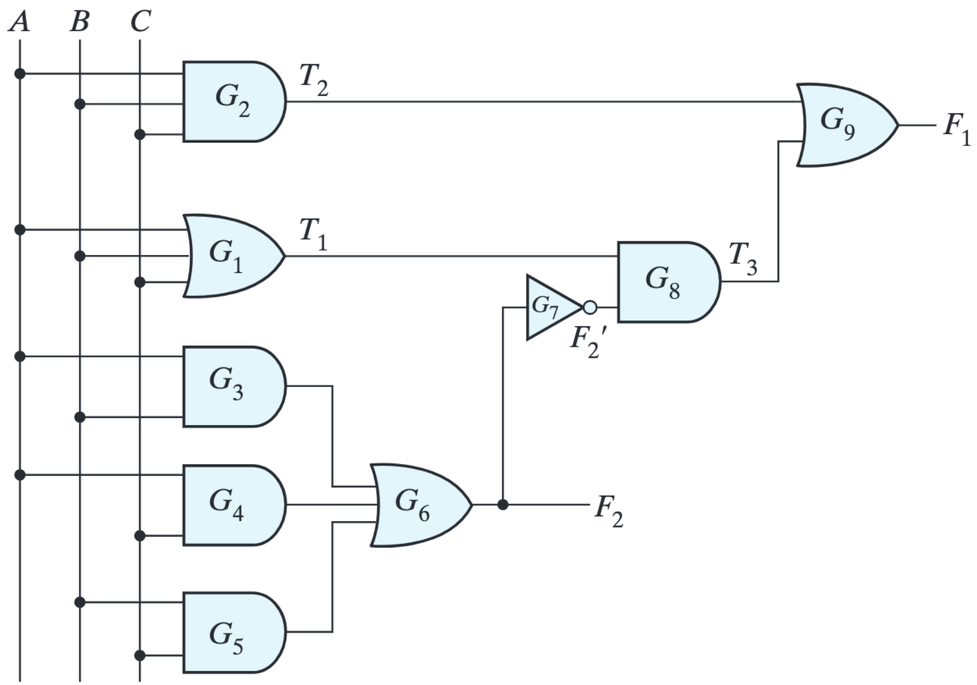


Analysis of combinational circuits

What truth table does the circuit implement?

- Go from input to output: Create truth table for intermediate variables
- Use them to write truth tables for next level

A	B	C	F_2	F_2'	T_1	T_2	T_3	F_1
0	0	0	0	1	0	0	0	0
0	0	1	0	1	1	0	1	1
0	1	0	0	1	1	0	1	1
0	1	1	1	0	1	0	0	0
1	0	0	0	1	1	0	1	1
1	0	1	1	0	1	0	0	0
1	1	0	1	0	1	0	0	0
1	1	1	1	0	1	1	0	1

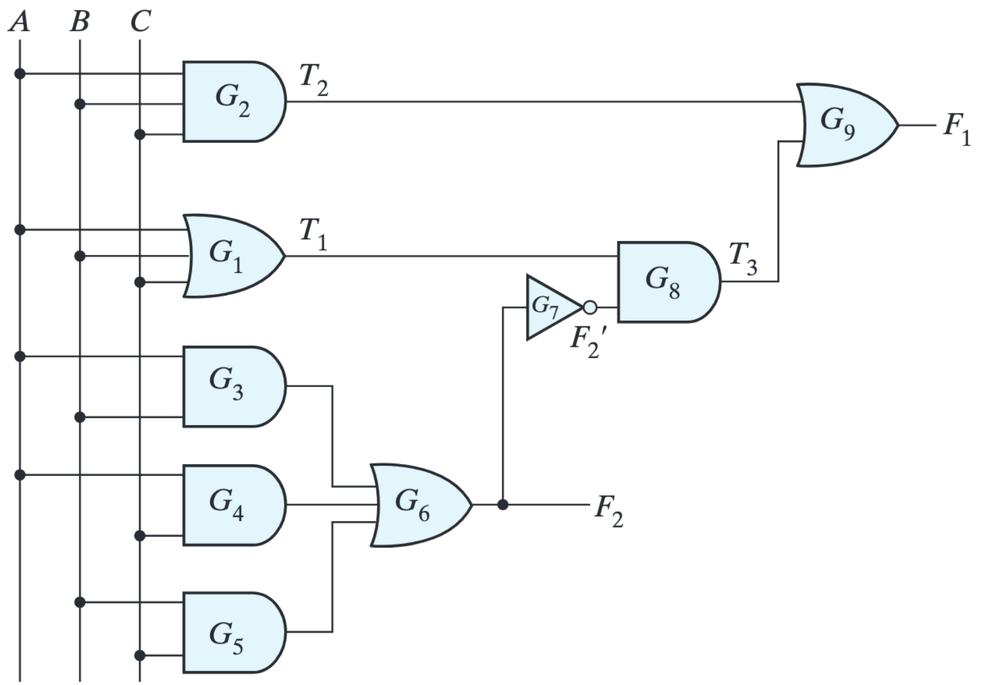


Analysis of combinational circuits

What does the circuit *mean*?

- Tough: Need to see many circuits to learn to identify patterns
- $A + B + C = F_2F_1$. Check with truth table

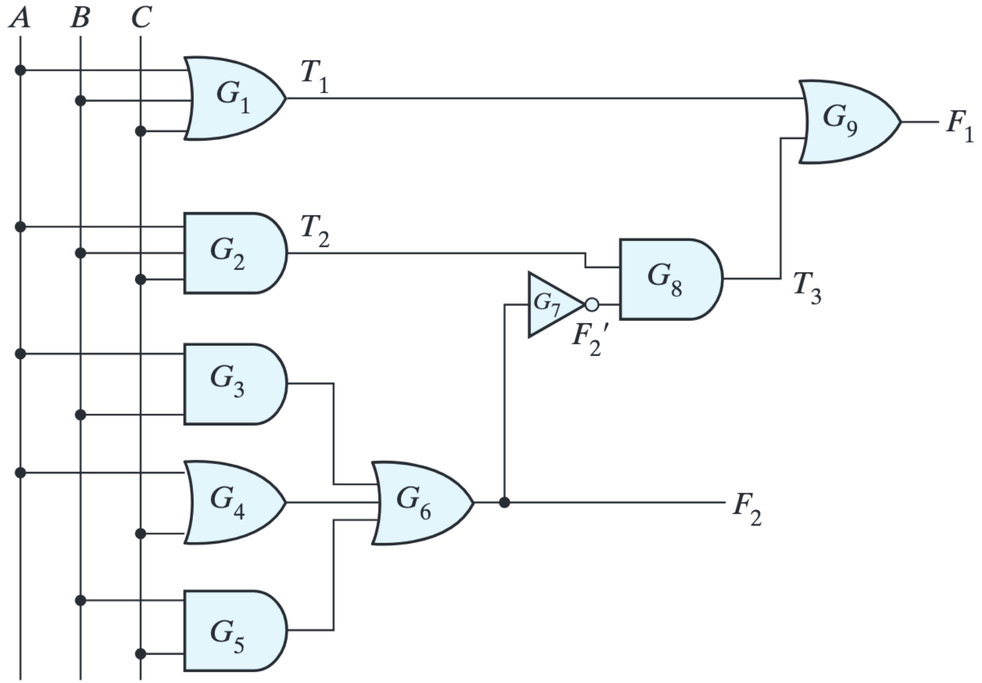
A	B	C	F_2	F_2'	T_1	T_2	T_3	F_1
0	0	0	0	1	0	0	0	0
0	0	1	0	1	1	0	1	1
0	1	0	0	1	1	0	1	1
0	1	1	1	0	1	0	0	0
1	0	0	0	1	1	0	1	1
1	0	1	1	0	1	0	0	0
1	1	0	1	0	1	0	0	0
1	1	1	1	0	1	1	0	1



What functions does this circuit implement?

Was a simpler circuit possible?

- $T_1 = A + B + C$
- $T_2 = ABC$
- $F_2 = AB + A + B + BC = A + B + BC = A + B$
- $F'_2 = A'B'$
- $T_3 = (ABC)(A'B') = 0$
- $F_1 = T_1 = A + B + C$
- Much simpler circuit is possible:
  - $F_2 = A + B$
  - $F_1 = F_2 + C$



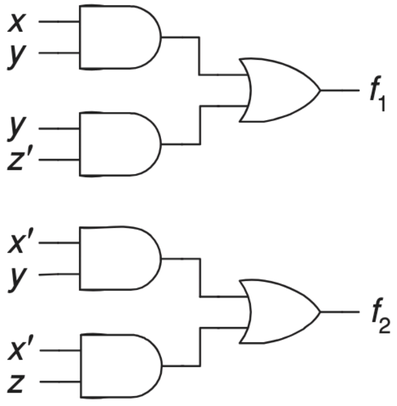
Designing combinational circuits

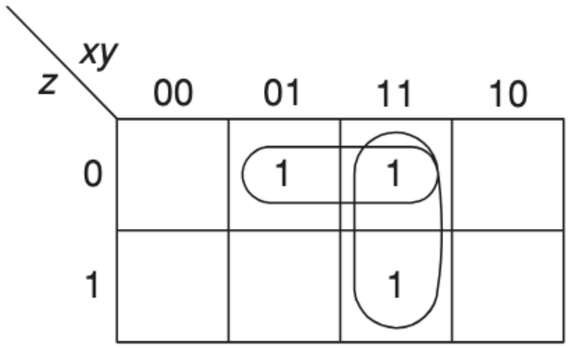
- Assign a symbol to each input and output
- Derive the truth table to find relationships between inputs and outputs
- Obtain simplified Boolean function for each output as a function of the input variables
- Reuse circuit elements between Boolean functions
- Draw the logic diagram and verify the correctness (manually or using simulation)

Reusing circuit elements between Boolean functions for two-level sum-of-products form

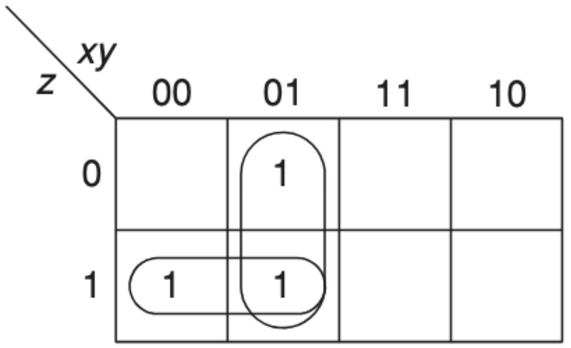
- $f_1 = \Sigma(2,6,7), f_2 = \Sigma(1,2,3)$
- Simplify: $f_1 = xy + yz' \quad f_2 = x'y + x'z$
- xy and yz' are prime implicants of f_1
  $x'y$ and $x'z$ are prime implicants of f_2

Function	Prime implicant	f_1 2	f_1 ✓6	f_1 ✓7	f_2 ✓1	f_2 2	f_2 ✓3
f_1	✓A = xy		×	×			
	B = yz'	×	×				
f_2	C = $x'y$					×	×
	✓D = $x'z$				×		×





(a) f_1.

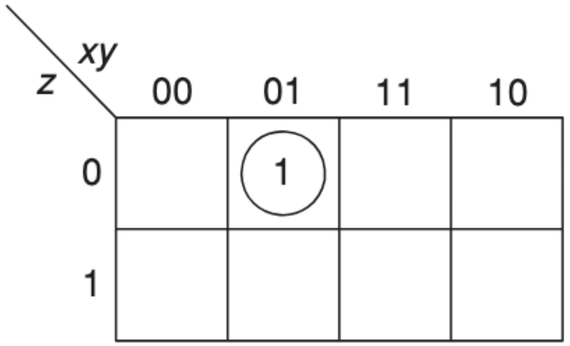


(b) f_2.

Reusing circuit elements between Boolean functions for two-level sum-of-products form

- $f_1 = \Sigma(2,6,7), f_2 = \Sigma(1,2,3)$
- $f_1 f_2 = \Sigma(2)$. All the implicants of $f_1 f_2$ are implicants of f_1 and of f_2
  $x'yz'$ is an implicant of both f_1 and f_2

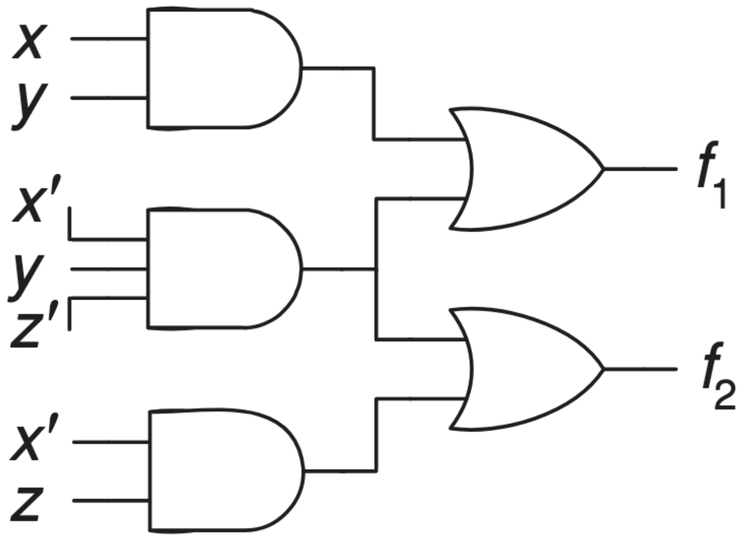
Function	Prime implicant	f_1 2	f_1 ✓ 6	f_1 ✓ 7	f_2 ✓ 1	f_2 2	f_2 ✓ 3
f_1	✓ A = xy		×	×			
	B = yz'	×	×				
f_2	C = $x'y$					×	×
	✓ D = $x'z$				×		×
$f_1 f_2$	E = $x'yz'$	×				×	



(c) $f_1 f_2$.

Reusing circuit elements between Boolean functions for two-level sum-of-products form

- $f_1 = \Sigma(1,3,7), f_2 = \Sigma(1,4,5)$
- Removing non-essential prime implicants B and C



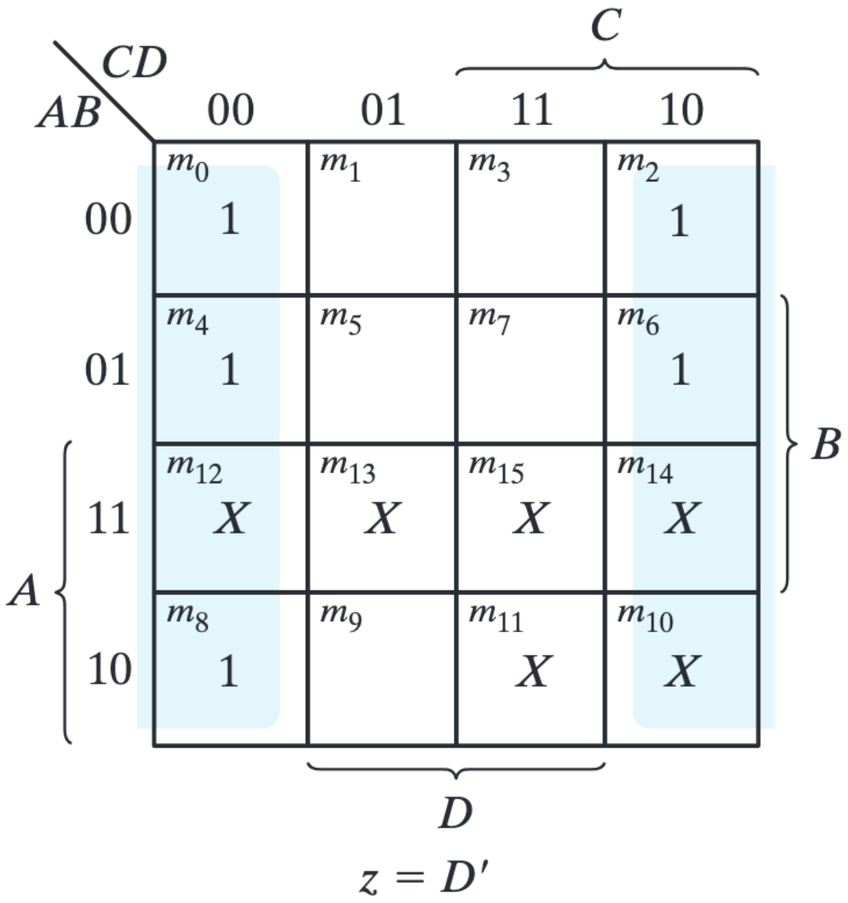
Function	Prime implicant	f_1 2	f_1 ✓6	f_1 ✓7	f_2 ✓1	f_2 2	f_2 ✓3
f_1	✓A = xy		×	×			
	B = yz'	×	×				
f_2	C = $x'y$					×	×
	✓D = $x'z$				×		×
$f_1 f_2$	E = $x'yz'$	×				×	

Reusing circuit elements between Boolean functions for multi-level circuits

- Consider this example
- $wxyz = ABCD + 3$

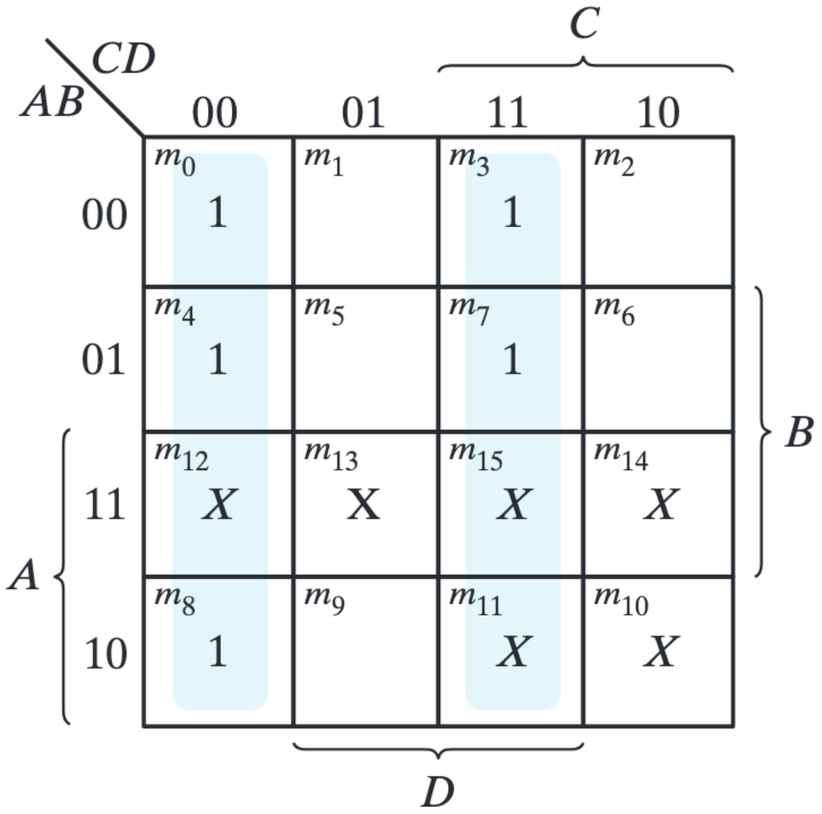
Input BCD				Output Excess-3 Code			
A	*B*	*C*	*D*	*w*	*x*	*y*	*z*
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0

Reusing circuit elements between Boolean functions for multi-level circuits (2)



Input BCD				Output Excess-3 Code			
A	*B*	*C*	*D*	*w*	*x*	*y*	*z*
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0

Reusing circuit elements between Boolean functions for multi-level circuits (3)

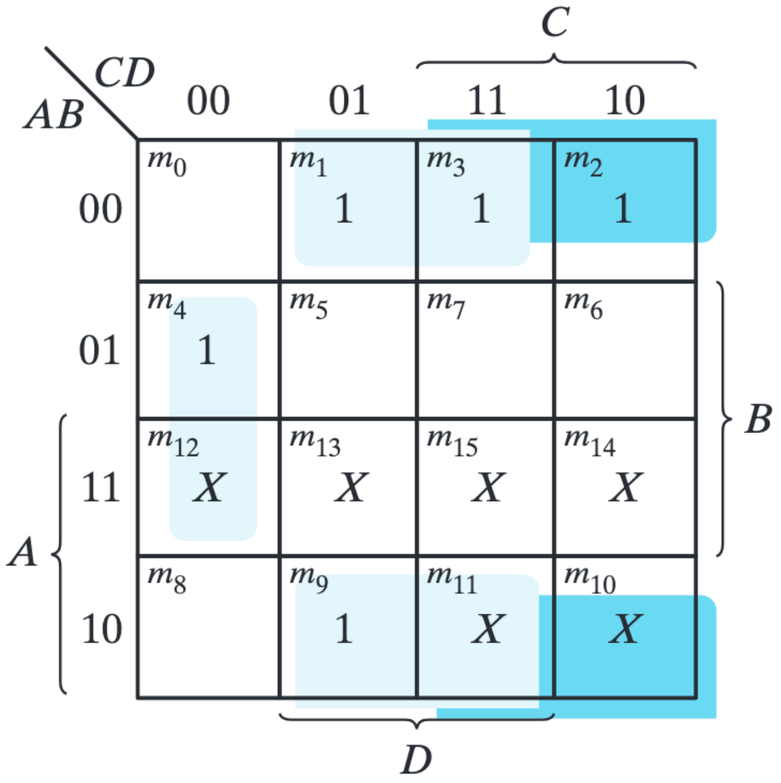


$$y = CD + C'D'$$

2 AND gates. 1 OR gate

Input BCD				Output Excess-3 Code			
A	*B*	*C*	*D*	*w*	*x*	*y*	*z*
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0

Reusing circuit elements between Boolean functions for multi-level circuits (4)

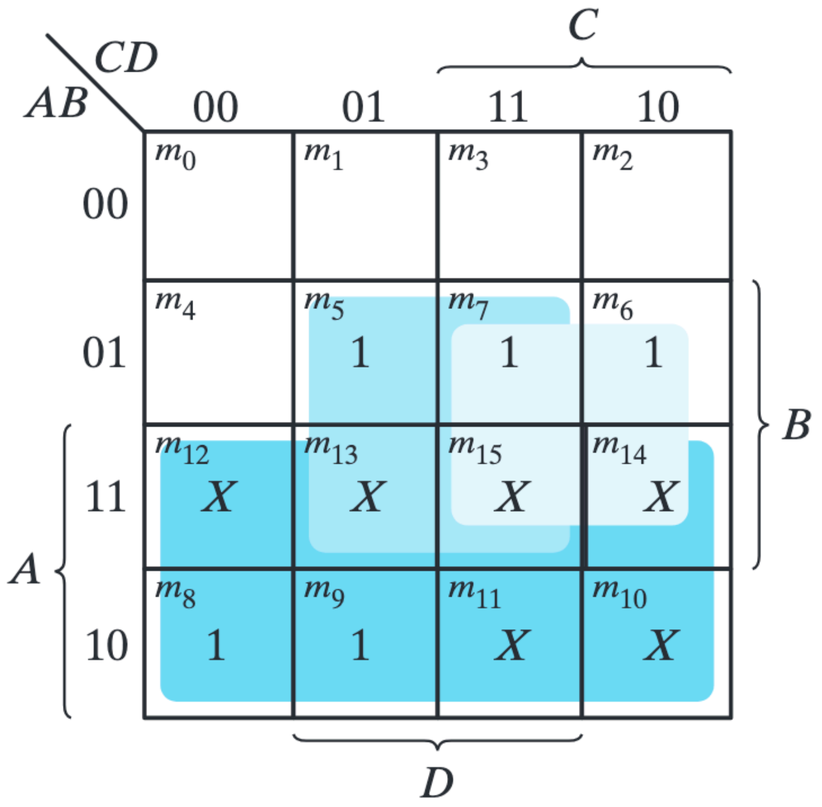


$$x = B'C + B'D + BC'D'$$

3 AND gates. 1 OR gate

Input BCD				Output Excess-3 Code			
A	*B*	*C*	*D*	*w*	*x*	*y*	*z*
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0

Reusing circuit elements between Boolean functions for multi-level circuits (5)



$$w = A + BC + BD$$

2 AND gates. 1 OR gate

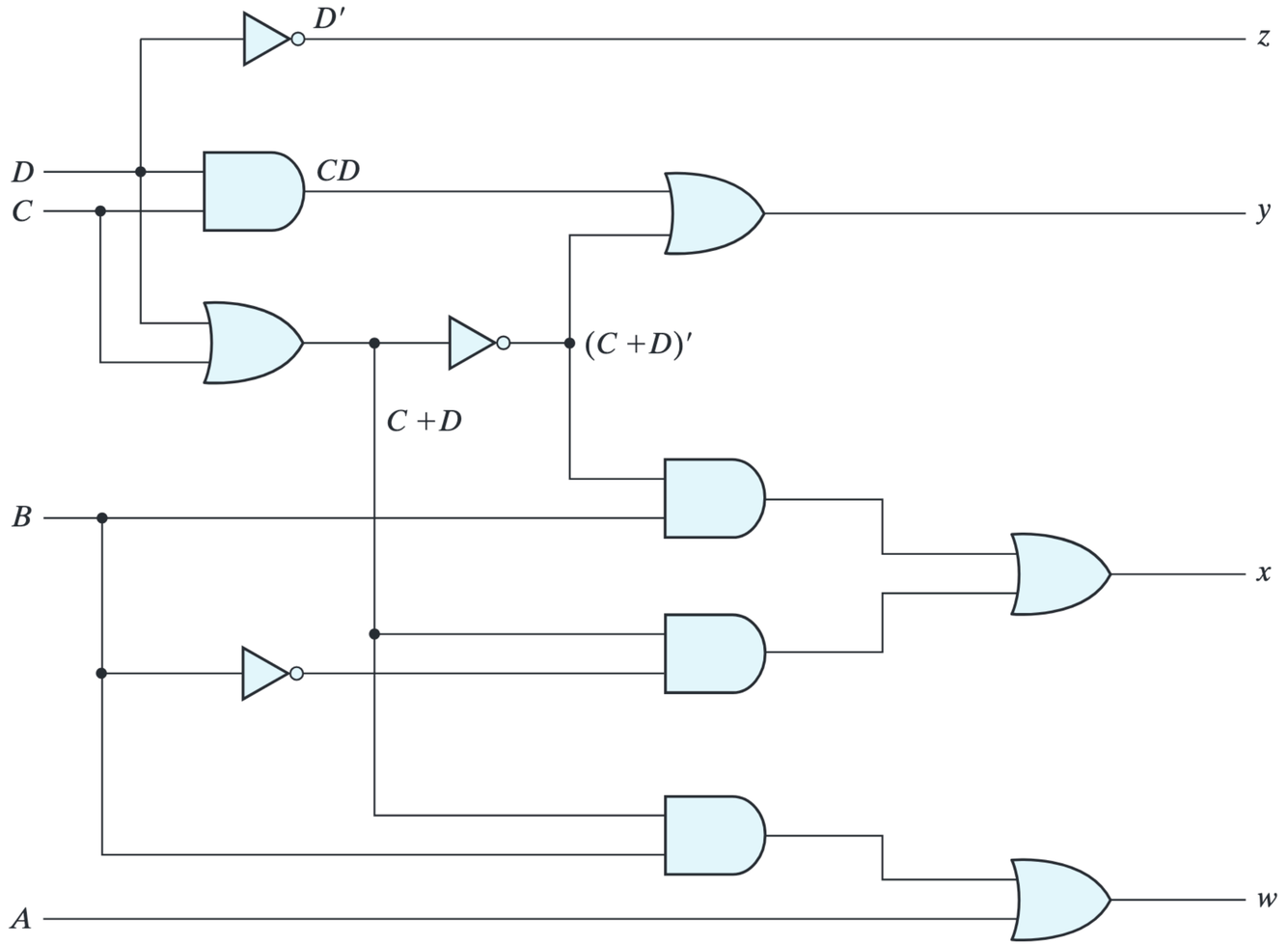
Input BCD				Output Excess-3 Code			
A	*B*	*C*	*D*	*w*	*x*	*y*	*z*
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0

Reusing circuit elements between Boolean functions for multi-level circuits (6)

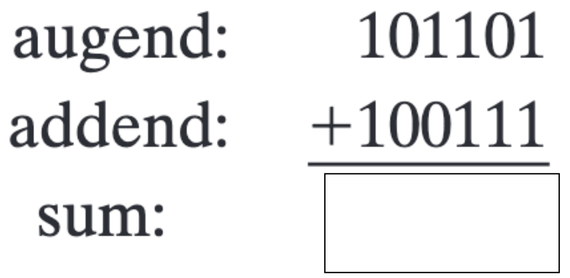
- 7 AND gates. 3 OR gates
- $z = D'$
- $y = CD + C'D' = CD + (C + D)'$
- $x = B'C + B'D + BC'D' = B'(C + D) + BC'D'$
  $= B'(C + D) + B(C + D)'$
- $w = A + BC + BD = A + B(C + D)$

Reusing circuit elements between Boolean functions for multi-level circuits (7)

- Consider this example
- $z = D'$
- $y = CD + (C + D)'$
- $x = B'(C + D) + B(C + D)'$
- $w = A + B(C + D)$
- 4 AND gates. 4 OR gates
- Logic synthesis tools automatically find and exploit subcircuits that are used by multiple outputs.



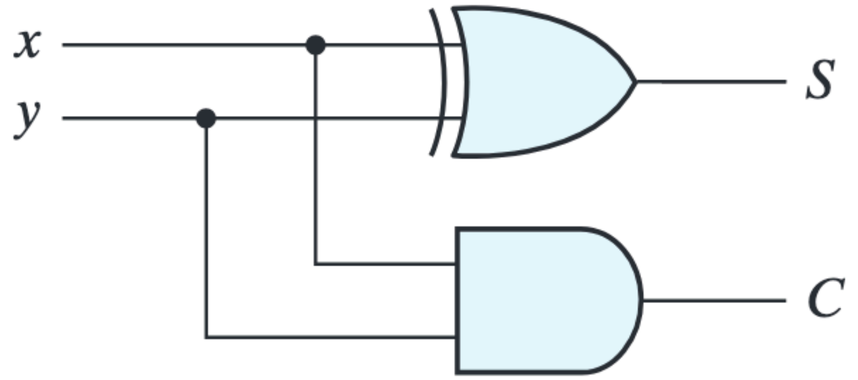
Binary Adder



- Most basic arithmetic operations. Multiplier uses adder circuit
- Bitwise adder: 0+0=0; 0+1=1; 1+0=1; 1+1=10 Carry
  - Adding two 1-bits, generate 2 output bits
- Binary adder: Carry obtained from adding bits in position, needs to be added to the bits in the next position
  - Half-adder: add two bits
  - Full-adder: add two bits and a carry. Can use two half-adders to build a full-adder

Half adder

- $S = x'y + xy'$
- $C = xy$



Half Adder

x	Y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Full adder

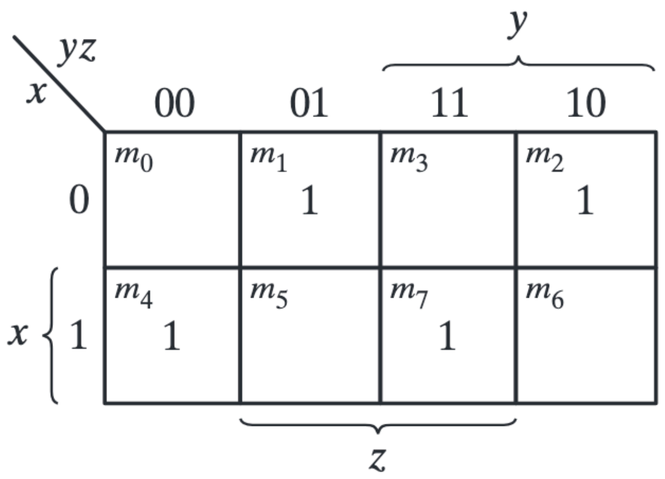
augend:	101101
addend:	+100111
sum:	1010100

- Addition of n-bit binary numbers proceeds bit-by-bit right-to-left
- Addition at each position adds two bits (x and y), and a carry (z) from preceding position
- Output is a bit (S) and a carry (C)

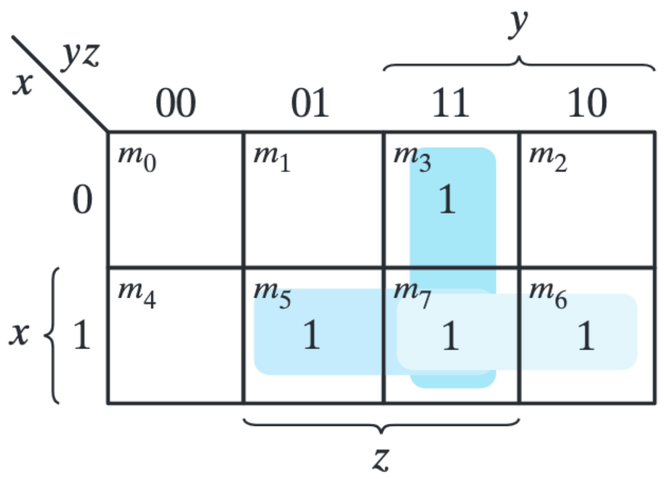
Full Adder

x	*y*	*z*	*C*	*S*
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

K-map for full-adder



(a) $S = x'y'z + x'yz' + xy'z' + xyz$



(b) $C = xy + xz + yz$

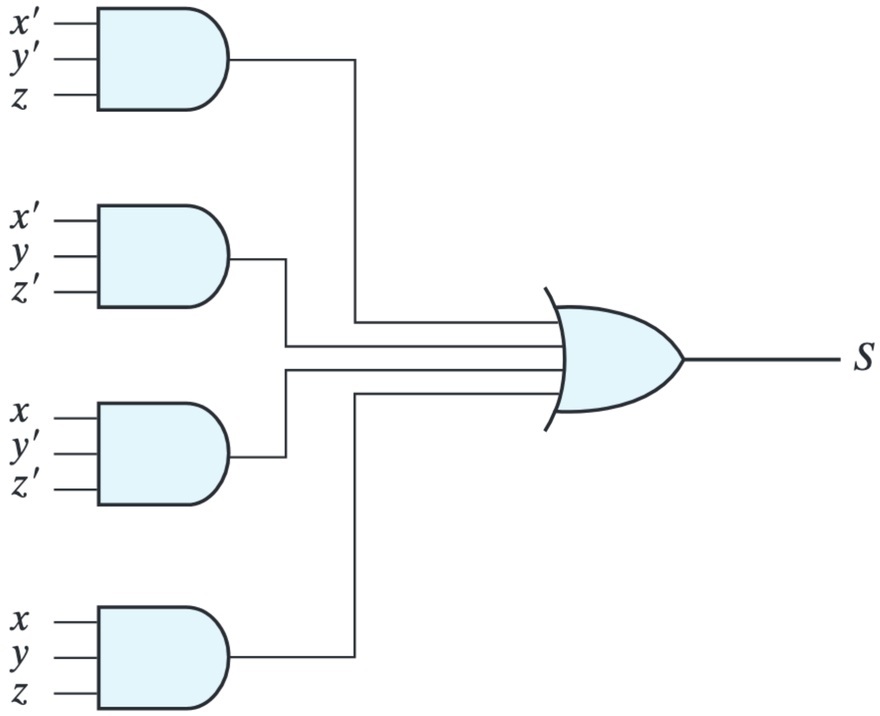
Full Adder

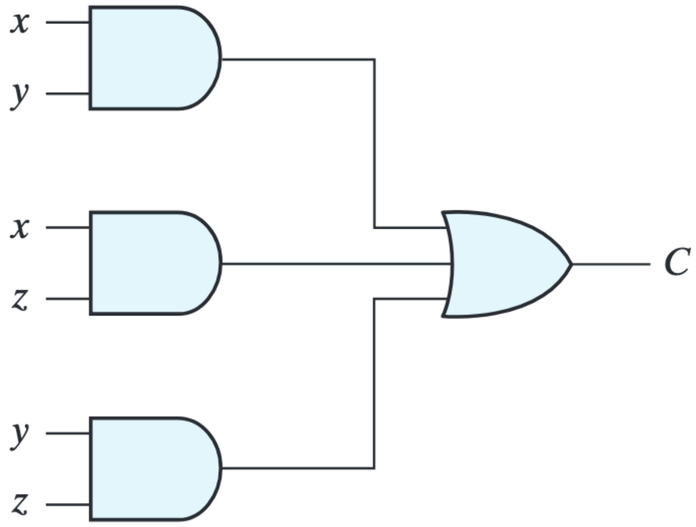
x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Full-adder implementation

Two-level implementation

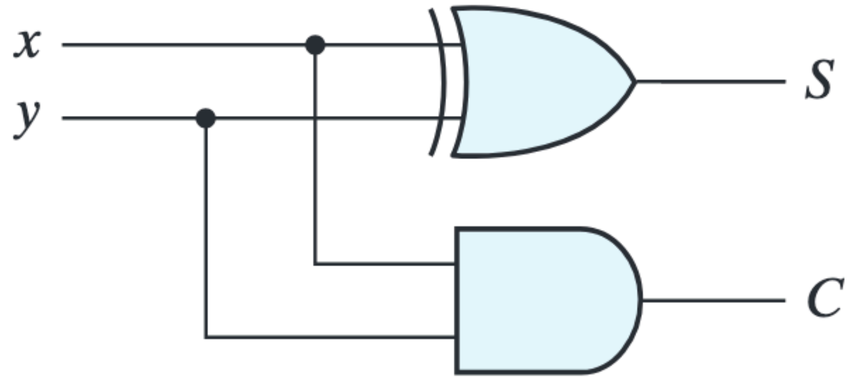
- $S = x'y'z + x'yz' + xy'z' + xyz$
- $C = xy + xz + yz$





Full-adder implementation with half-adders

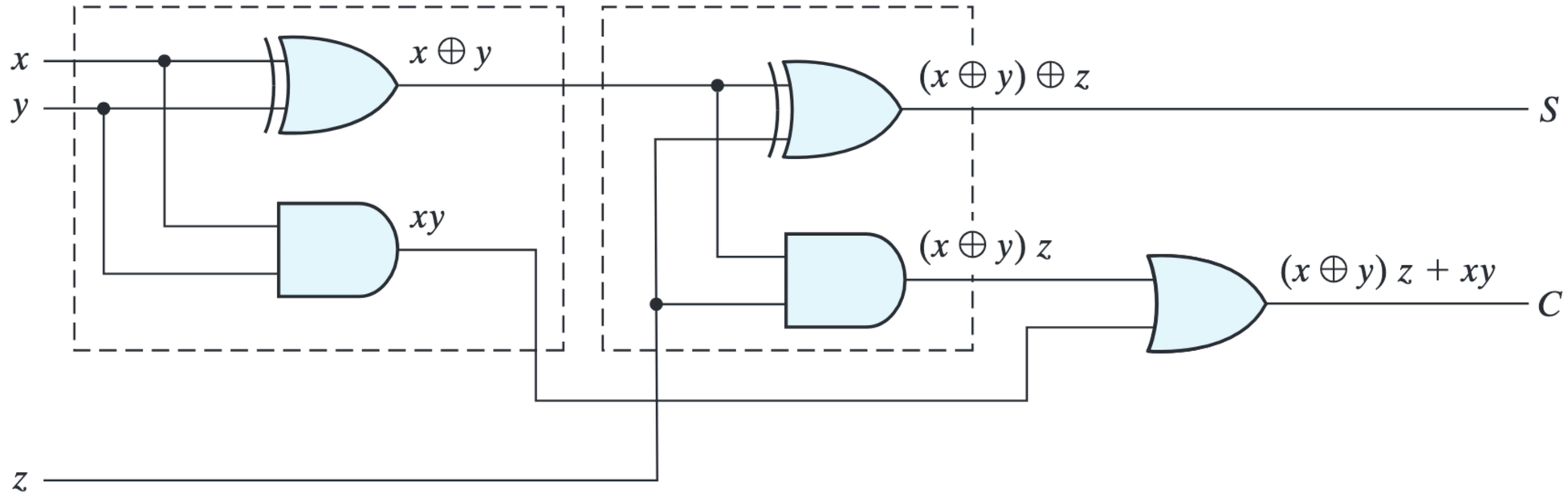
- $S = xy'z' + x'yz' + xyz + x'y'z$
- $S = z \oplus (x \oplus y)$
- $C = xy'z + x'yz + xy$
- $C = z(xy' + x'y) + xy = z(x \oplus y) + xy$



(b) $S = x \oplus y$
$C = xy$

Full-adder implementation with half-adders (2)

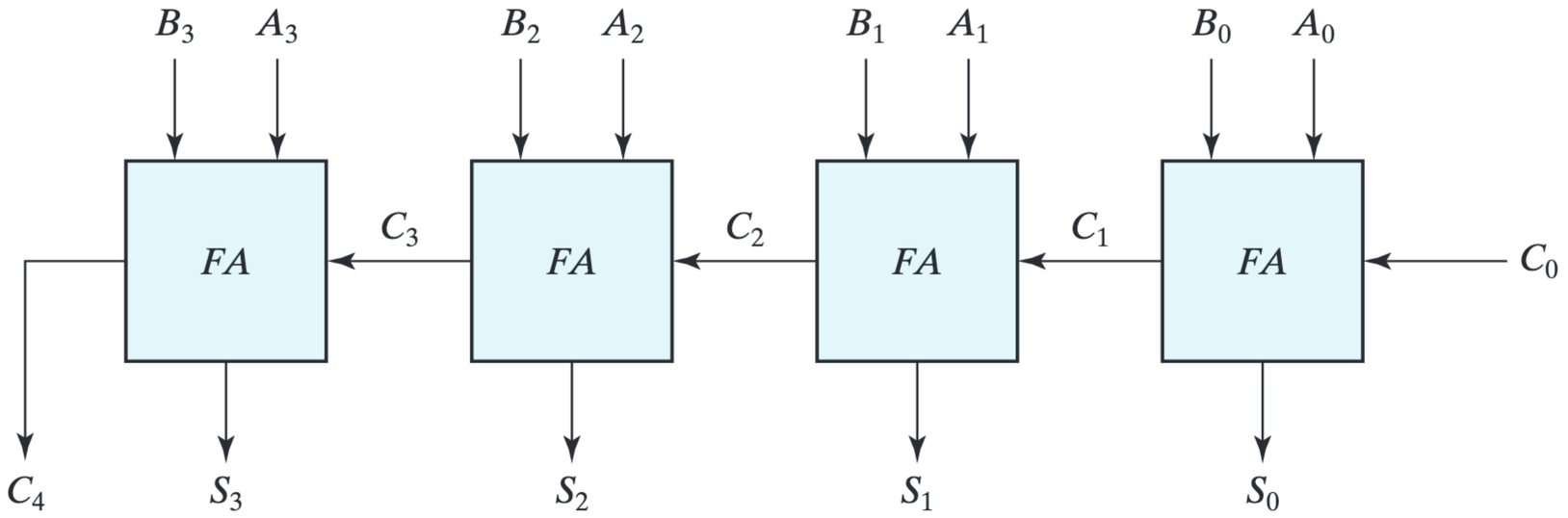
- $S = xy'z' + x'yz' + xyz + x'y'z$
- $S = z \oplus (x \oplus y)$
- $C = xy'z + x'yz + xy$
- $C = (xy' + x'y)z + xy = (x \oplus y)z + xy$



Binary Adder

- Full adders connected in cascade, with the output carry from each full adder connected to the input carry of the next full adder in the chain
- Carry bit *ripples* through the full adders

Subscript *i*:	3	2	1	0	
Input carry	0	1	1	0	C_i
Augend	1	0	1	1	A_i
Addend	0	0	1	1	B_i
Sum	1	1	1	0	S_i
Output carry	0	0	1	1	C_{i+1}



Four-bit adder

$C_4C_3C_2C_1C_0$

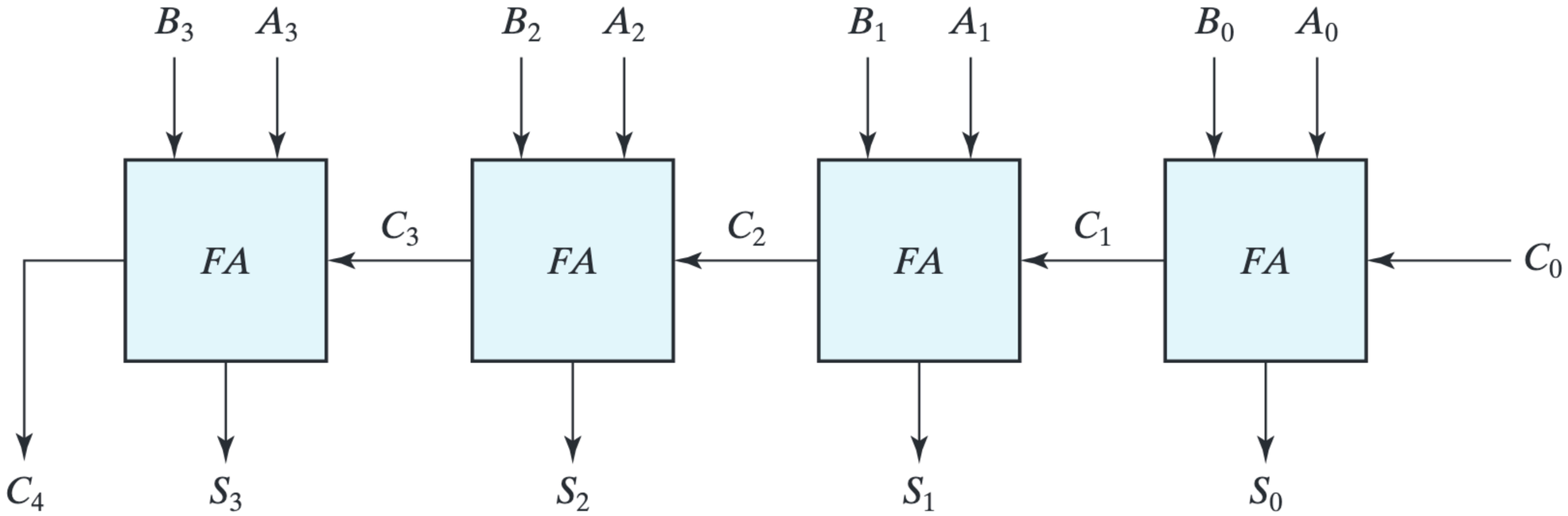
$A_3A_2A_1A_0$

$B_3B_2B_1B_0$

$S_3S_2S_1S_0$

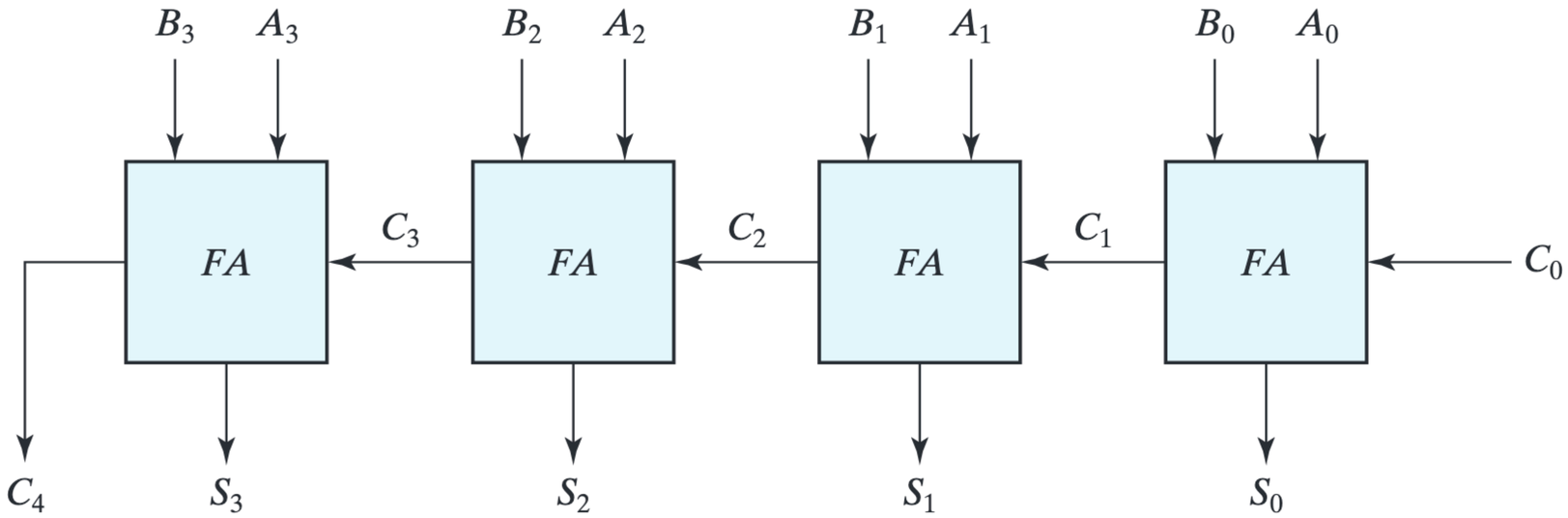
Overflow Detection

- Last carry bit detects an overflow
- Example: 10+7 overflows into 5 bits



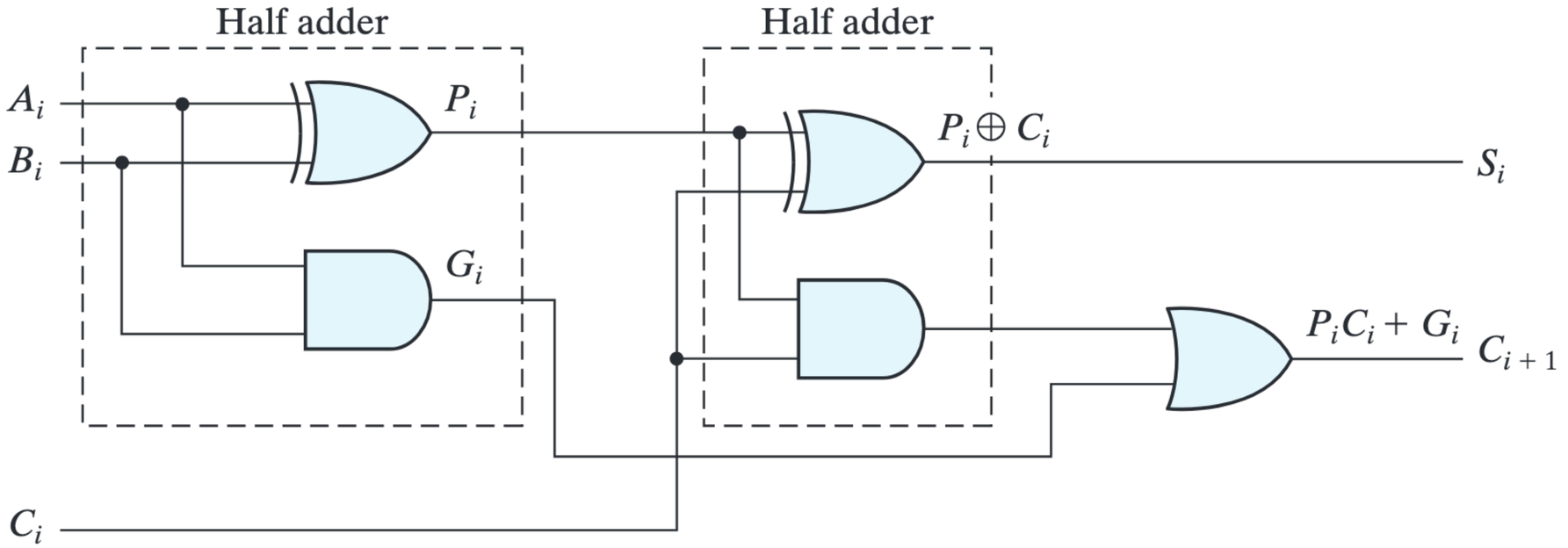
Problem with Ripple adder

- Adding N-bit numbers requires N full adders. Longest propagation delay time in an adder is the time it takes the carry to propagate through the full adders
- For computing S_3, A_3 and B_3 are available as soon as input signals are applied to the adder. However, input carry C_3 is not available
- C_3 does not settle to its final value until C_2 is available from the previous stage.



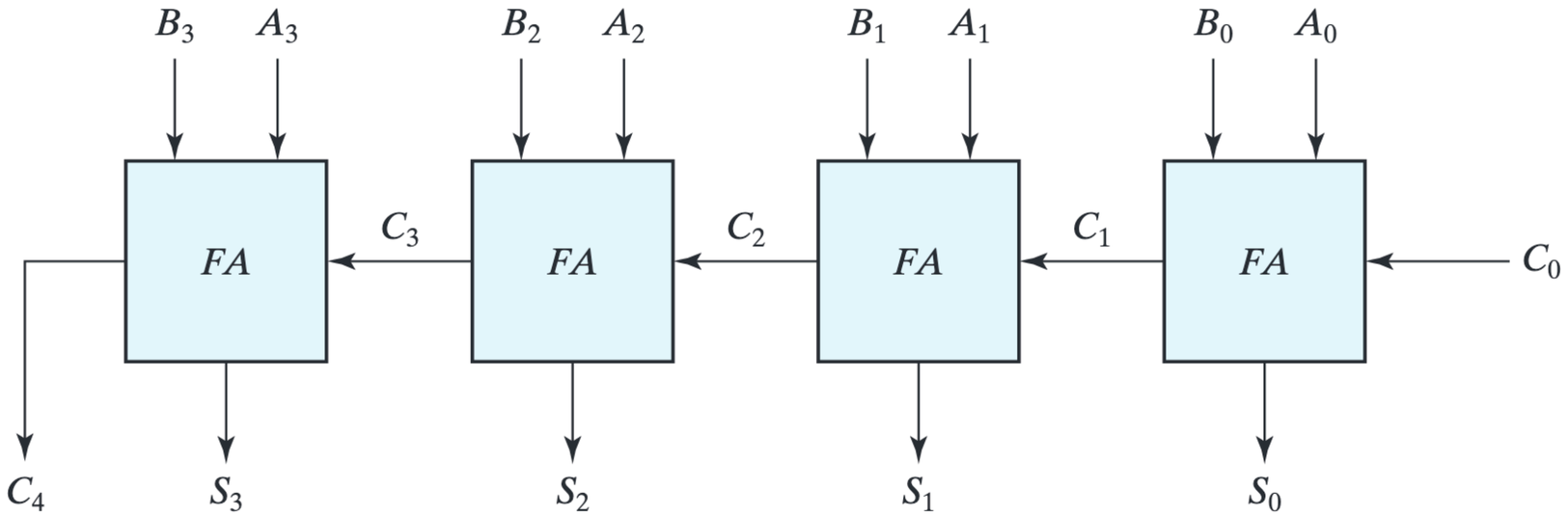
Carry propagation delay

- P_i and G_i are available after 1-gate's propagation delay after the inputs are applied
- C_{i+1} requires two gate delays from C_i: one AND and an OR.
  C_4 requires 2*4=8 gate delays to compute



Carry propagation delay

- Limits the speed with which two numbers can be added: For an n-bit adder, there are 2n gate levels for the carry to propagate
- Solution?
  - Employ faster gates with reduced delays. However, physical circuits have a limit to their capability.
  - Increase circuit complexity to reduce carry propagation delay



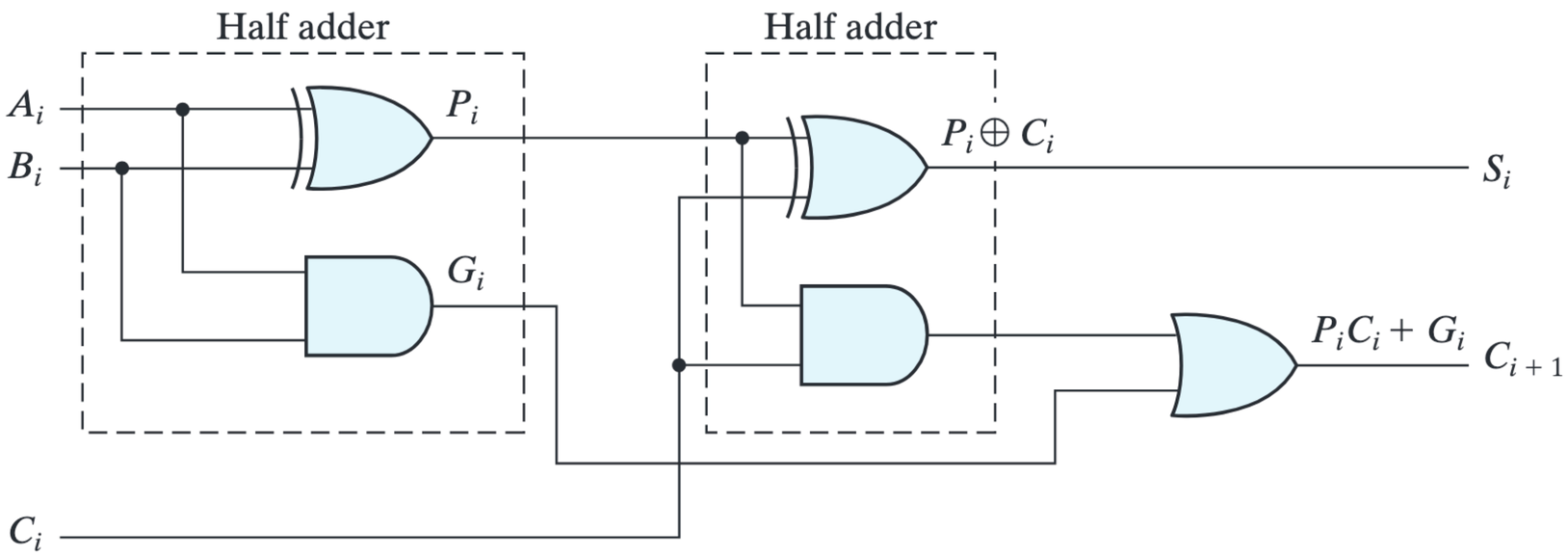
Carry lookahead logic

- $C_0 = input\ carry$
- $C_1 = G_0 + P_0C_0$
- $C_2 = G_1 + P_1C_1$

  $= G_1 + P_1(G_0 + P_0C_0)$

  $= G_1 + P_1G_0 + P_1P_0C_0$
- $C_3 = G_2 + P_2C_2$

  $= G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0$



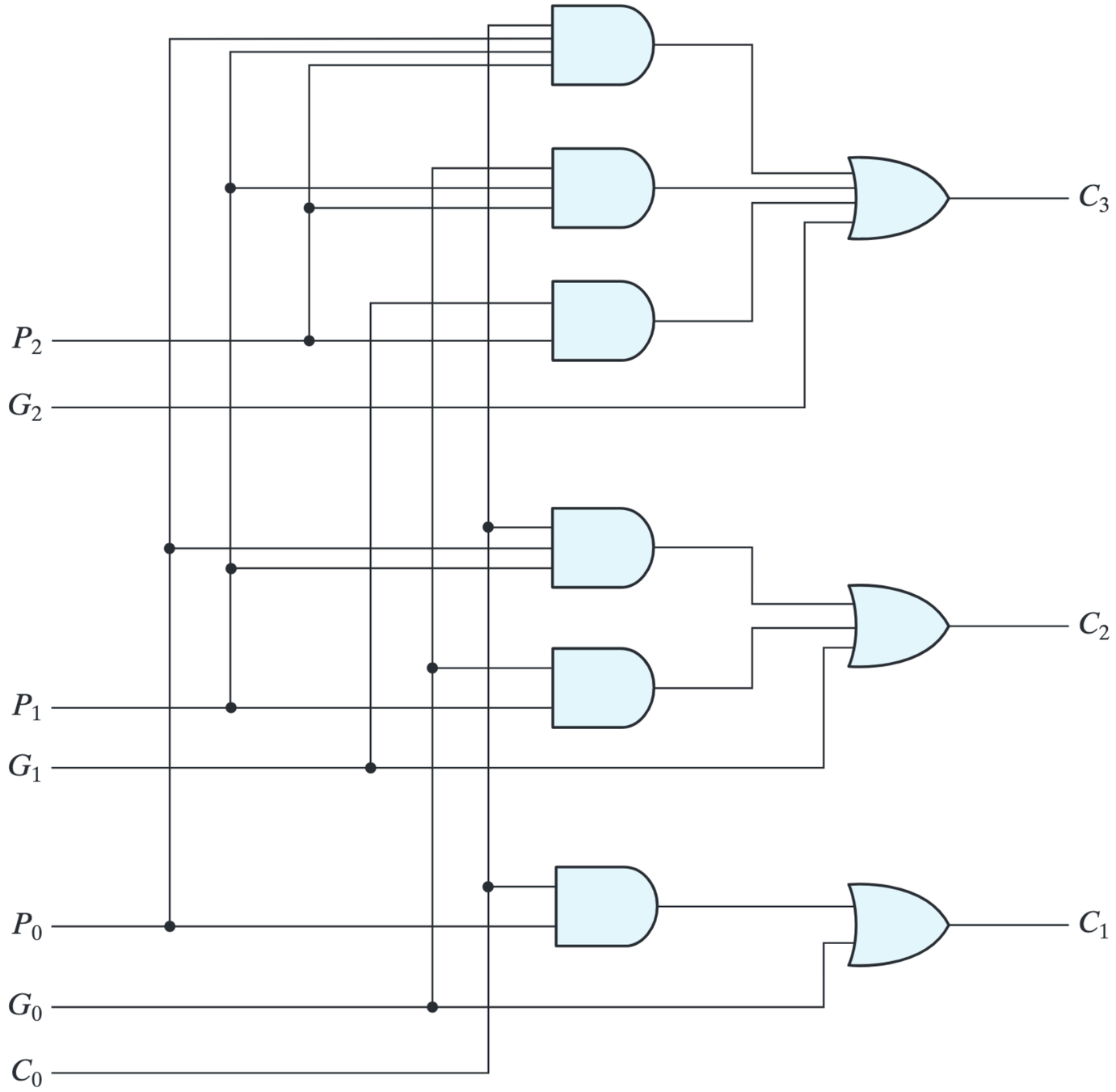
Carry lookahead generator

- $C_0 = input\ carry$
- $C_1 = G_0 + P_0C_0$
- $C_2 = G_1 + P_1C_1$

  $= G_1 + P_1(G_0 + P_0C_0)$

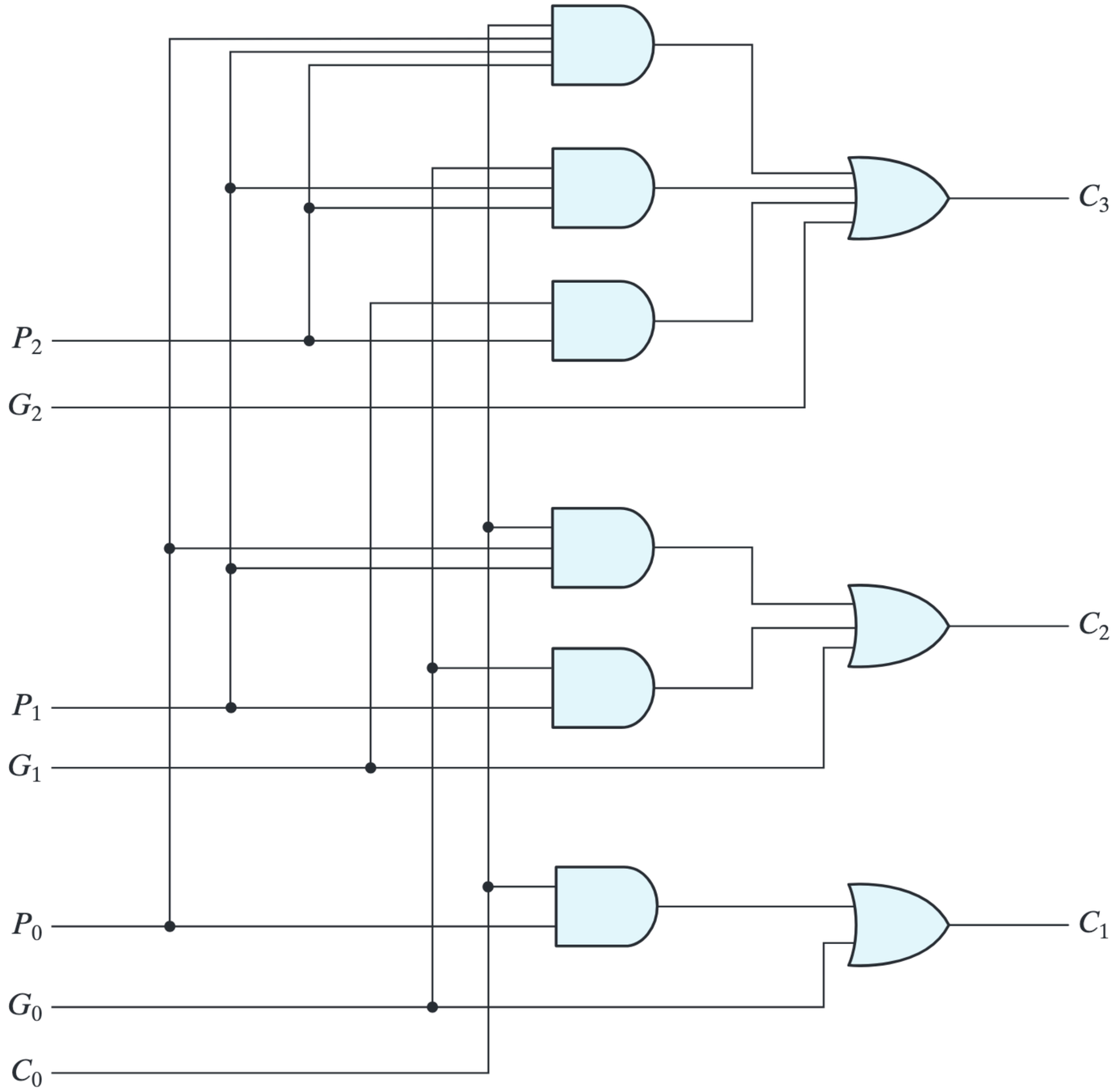
  $= G_1 + P_1G_0 + P_1P_0C_0$
- $C_3 = G_2 + P_2C_2$

  $= G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0$



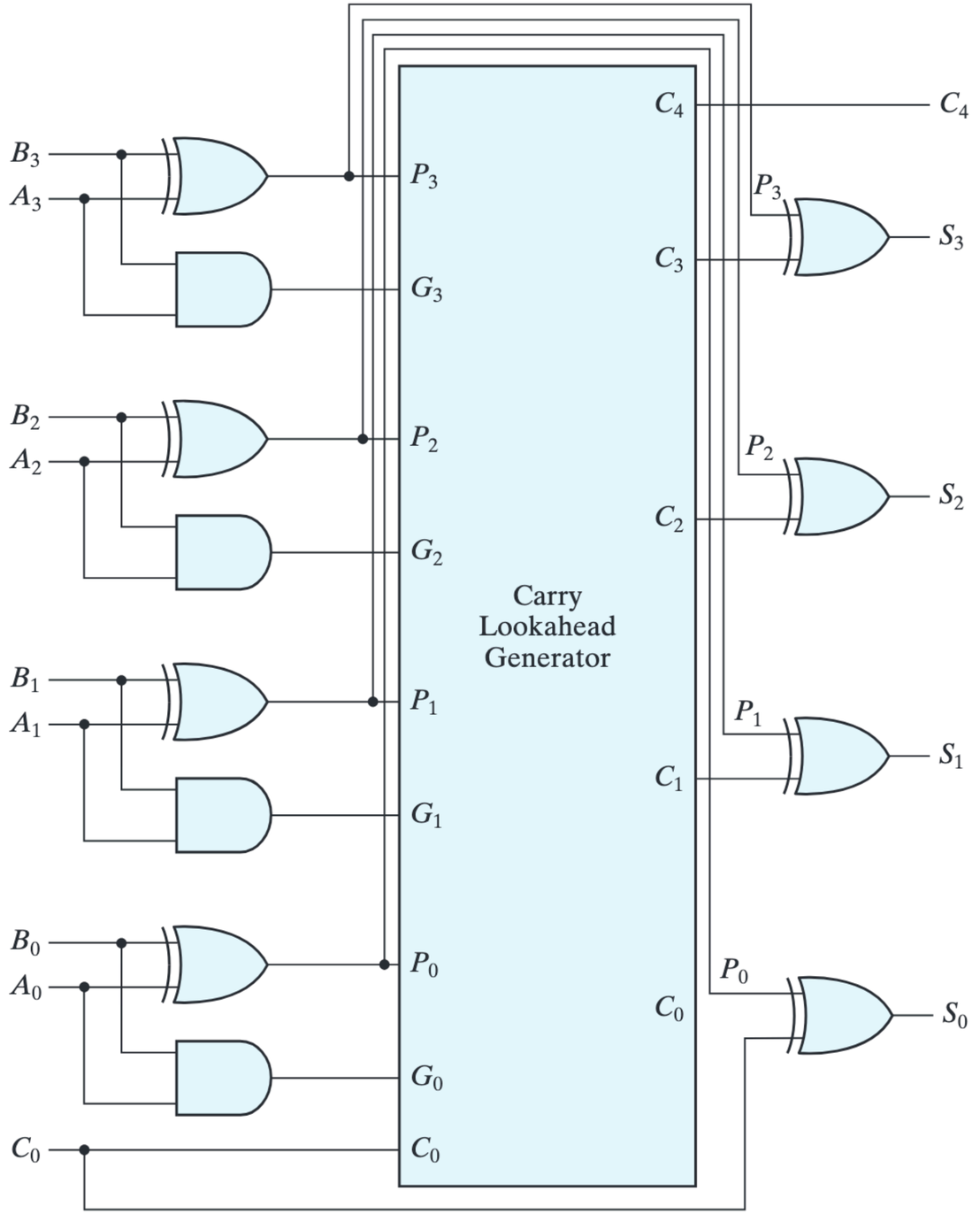
Carry lookahead generator

- C_3 does not have to wait for C_2
- All C_1, C_2, C_3 are generated in two gate delays
- Faster circuit but at the expense of more gates



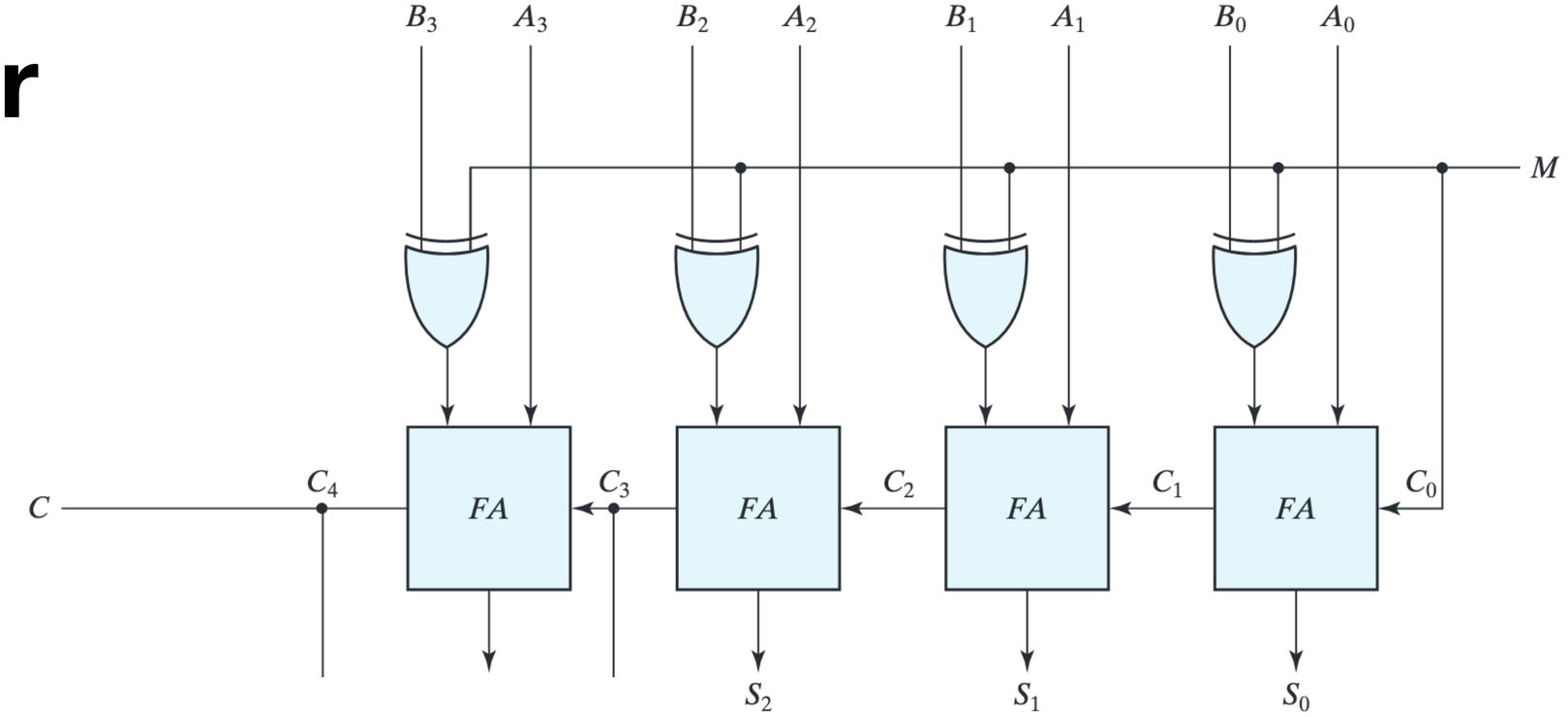
Binary adder with carry lookahead

- P_i and G_i are generated in 1 gate delay
- C_i is generated in two gate delays
- Therefore, all S_i bits are generated in 4 gate delays
- Ripple adder was generating S_n in $2n$ gate delays



Binary subtractor

- “A - B” is
  “A + (2’s complement of
- 2’s-complement of B is:
  - flip all bits of B.
    $b \oplus 1 = b'$
  - Add 1
- When M = 1, realises a binary subtractor. Flip all bits of B, $C_0 = 1$



- When M = 0, realises a binary adder.
  $b \oplus 0 = b$, $C_0 = 0$

Overflow in two's complement representation

- Overflow cannot occur from addition of a positive number (x) and a negative number (y)
  - Produces a result that is smaller than x and larger than y
- Overflow occurs when adding two positive numbers or two negative numbers.
  - 70+80 overflows 8 bits.
    - Sets the "sign bit" to 1 and leaves a carry of 0.
  - -70-80 overflows 8 bits.
    - Sets the "sign bit" to 0 and leaves a carry of 1

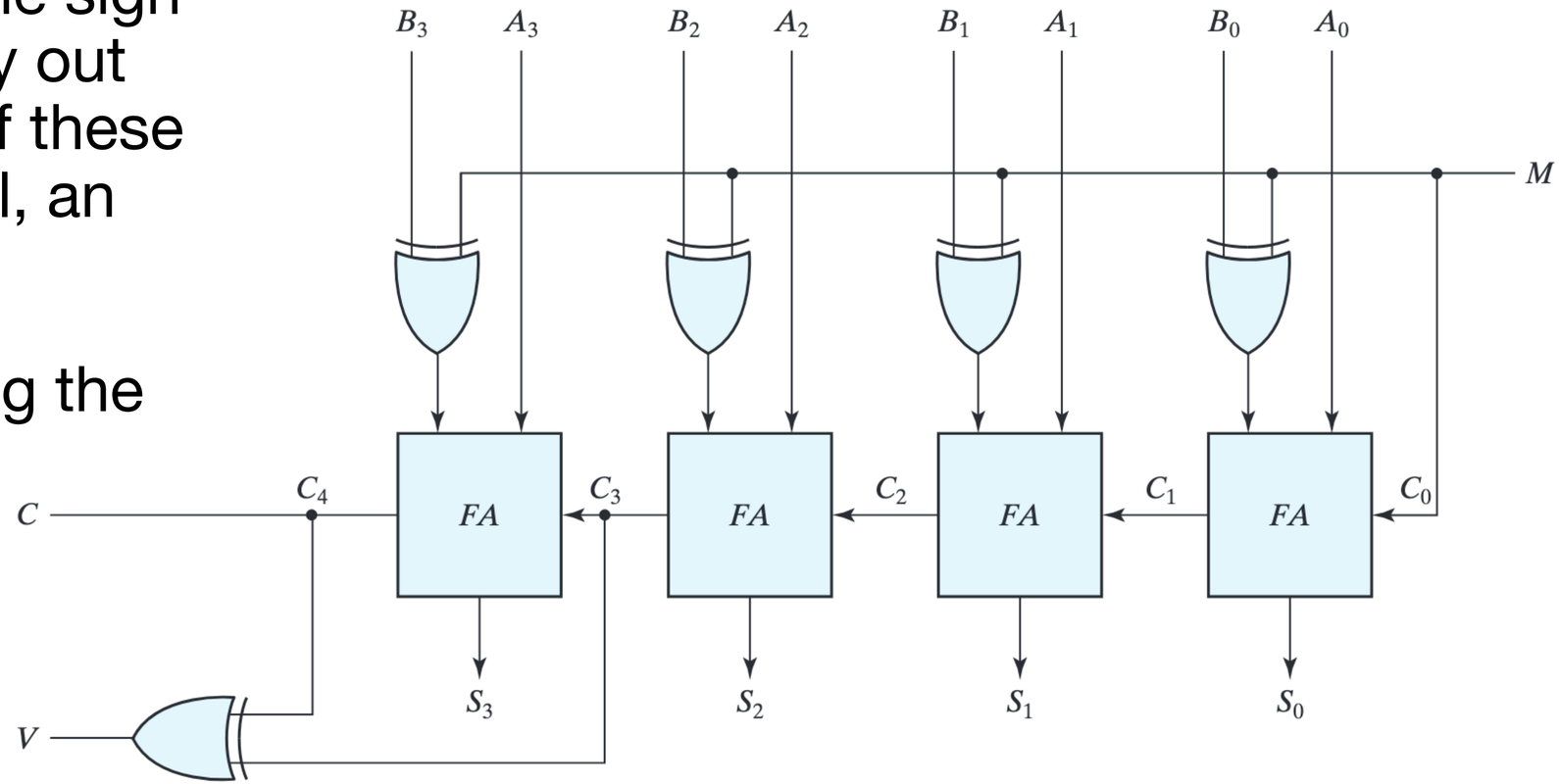
carries:	0 1
+70	0 1000110
+80	0 1010000
150	1 0010110

carries:	1 0
−70	1 0111010
−80	1 0110000
−150	0 1101010

Overflow detection

carries:	0 1	carries:	1 0
+70	0 1000110	−70	1 0111010
+80	0 1010000	−80	1 0110000
150	1 0010110	−150	0 1101010

- Observe the carry into the sign bit position and the carry out of the sign bit position. If these two carries are not equal, an overflow has occurred.
- Detect overflow by xoring the two carry bits

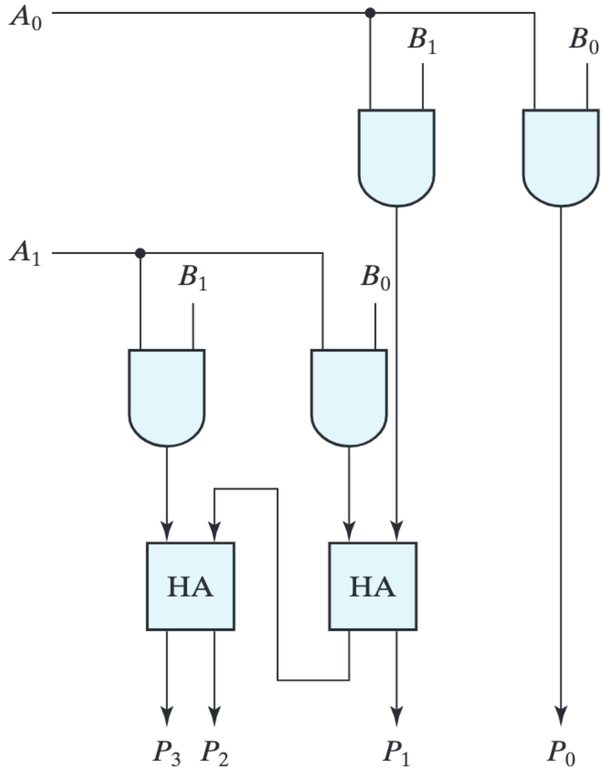


Binary multiplier

Two-bit multiplier

- Bit-wise multiplication is simply an AND: produce a 1 if both bits were 1
- Least significant bit P_0 is simply output of the AND gate
- $P_1 = A_0B_1 + A_1B_0$ is simply output of half-adder
- The carry goes to A_1B_1 to output P_2 and P_3

$$
\begin{array}{cccc}
 & & B_1 & B_0 \\
 & & A_1 & A_0 \\
\hline
 & & A_0B_1 & A_0B_0 \\
 & A_1B_1 & A_1B_0 & \\
\hline
P_3 & P_2 & P_1 & P_0
\end{array}
$$

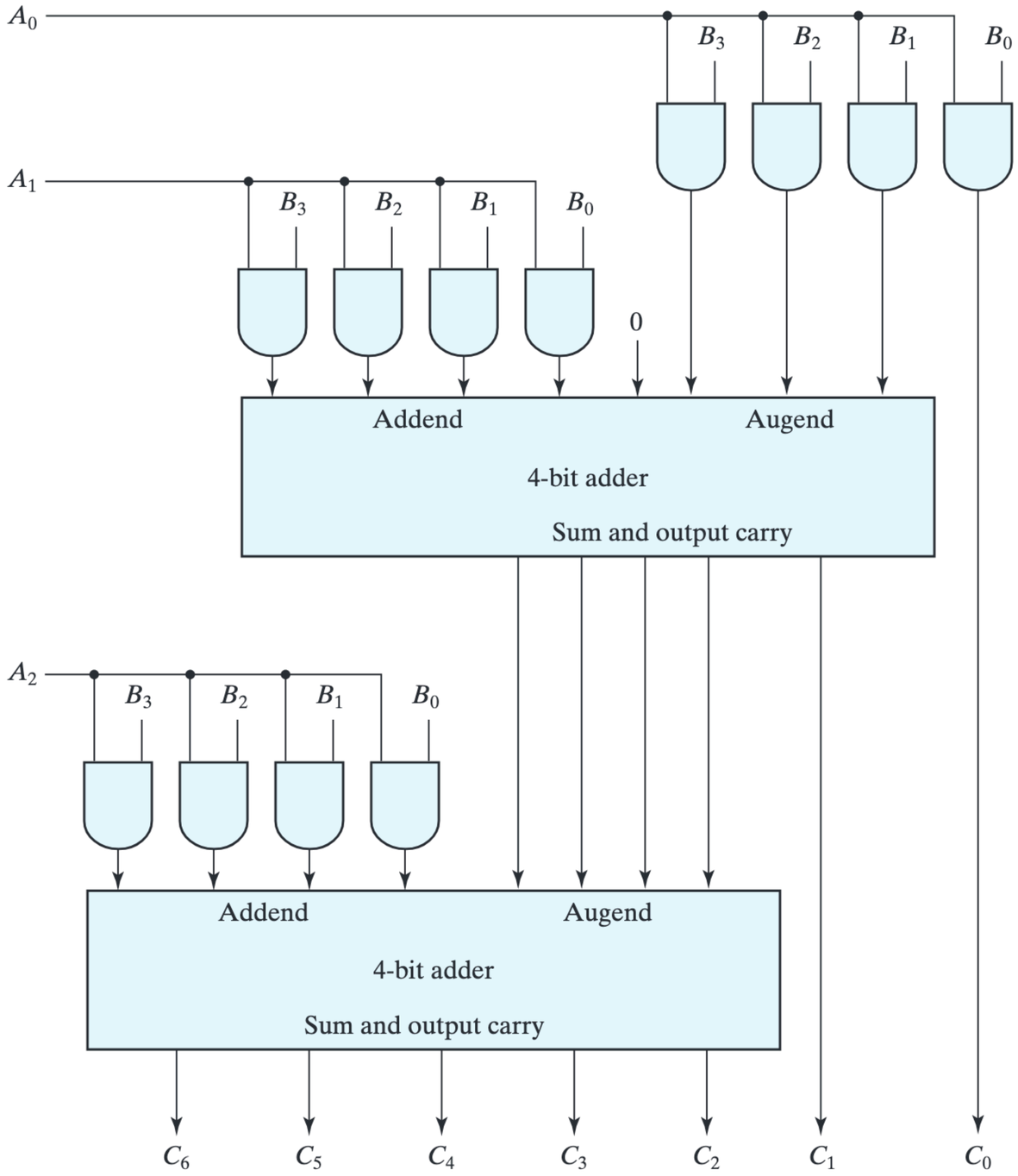


Binary multiplier

4 X 3 bit multiplier

- Least significant bit C_0 is simply output of the AND gate
- Output in each level of AND gates is added with the partial product of the previous level to form a new partial product

multiplicand:	1011
multiplier:	× 101
	1011
	0000
partial product:	1011
product:	110111

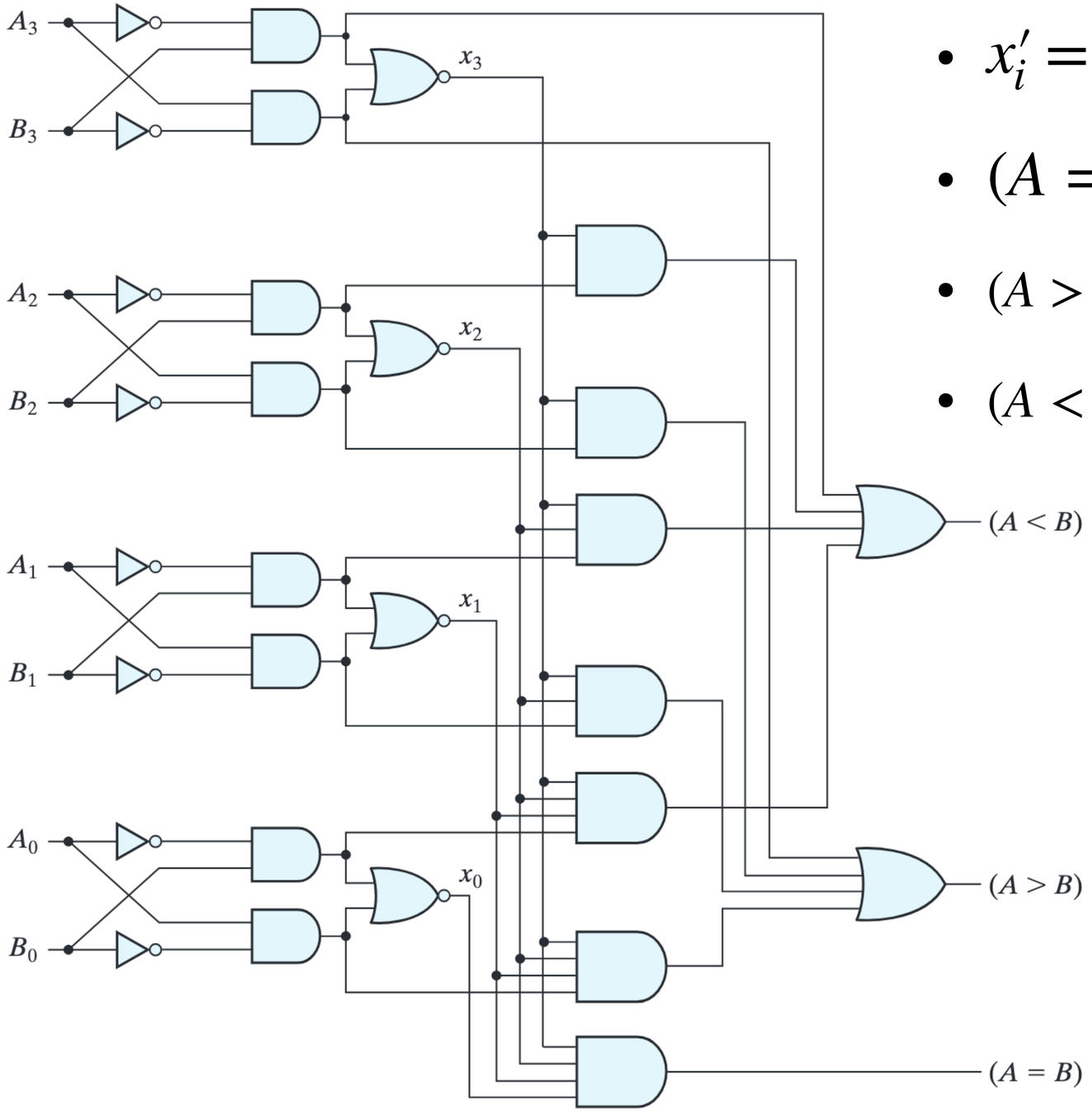


Magnitude Comparator

- Consider two binary numbers: $A = A_3A_2A_1A_0$ and $B = B_3B_2B_1B_0$. Comparator outputs three boolean functions $A = B$, $A < B$, and $A > B$
- $A = B$ if all bits are equal. $x_i = (A_i \oplus B_i)'$ or $x_i = A_iB_i + A_i'B_i'$ for $i = 0,1,2,3$ $(A = B) = x_3x_2x_1x_0$

Magnitude Comparator (2)

- $A > B$
  - if $A_3 > B_3$, (i.e., $A_3 B_3'$)
    - When $A_3 = B_3$ if $A_2 > B_2$ (i.e., $A_2 B_2'$)
      - When $A_2 = B_2$ if $A_1 > B_1$ (i.e., $A_1 B_1'$)
        - When $A_1 = B_1$ if $A_0 > B_0$ (i.e., $A_0 B_0'$)
- $(A > B) = A_3 B_3' + x_3 A_2 B_2' + x_3 x_2 A_1 B_1' + x_3 x_2 x_1 A_0 B_0'$
- $(A < B) = A_3' B_3 + x_3 A_2' B_2 + x_3 x_2 A_1' B_1 + x_3 x_2 x_1 A_0' B_0$



- $x_i' = A_i' B_i + A_i B_i'$
- $(A = B) = x_3 x_2 x_1 x_0$
- $(A > B) = A_3 B_3' + x_3 A_2 B_2' + x_3 x_2 A_1 B_1' + x_3 x_2 x_1 A_0 B_0'$
- $(A < B) = A_3' B_3 + x_3 A_2' B_2 + x_3 x_2 A_1' B_1 + x_3 x_2 x_1 A_0' B_0$

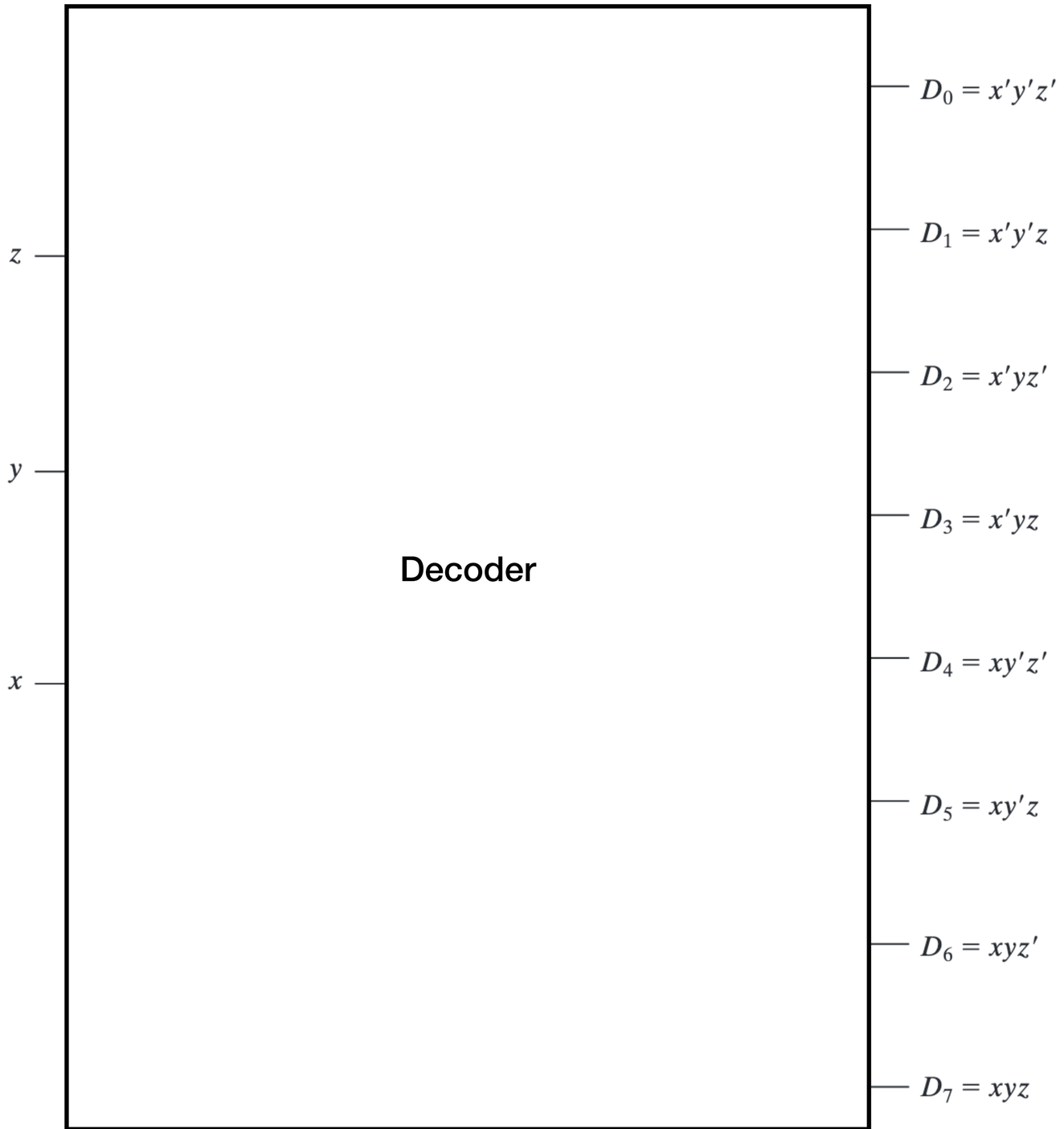
Magnitude Comparator (3)

Decoders

- Convert binary information from n input lines to a maximum of 2^n unique output lines. Only one output line is *active* at a time.
  - Given an n-bit address, memory needs to find the byte from $\{0,\ldots,2^{n-1}\}$
- n-to-m decoders
  - Given an instruction opcode, CPU needs to decode it and activate the right circuit

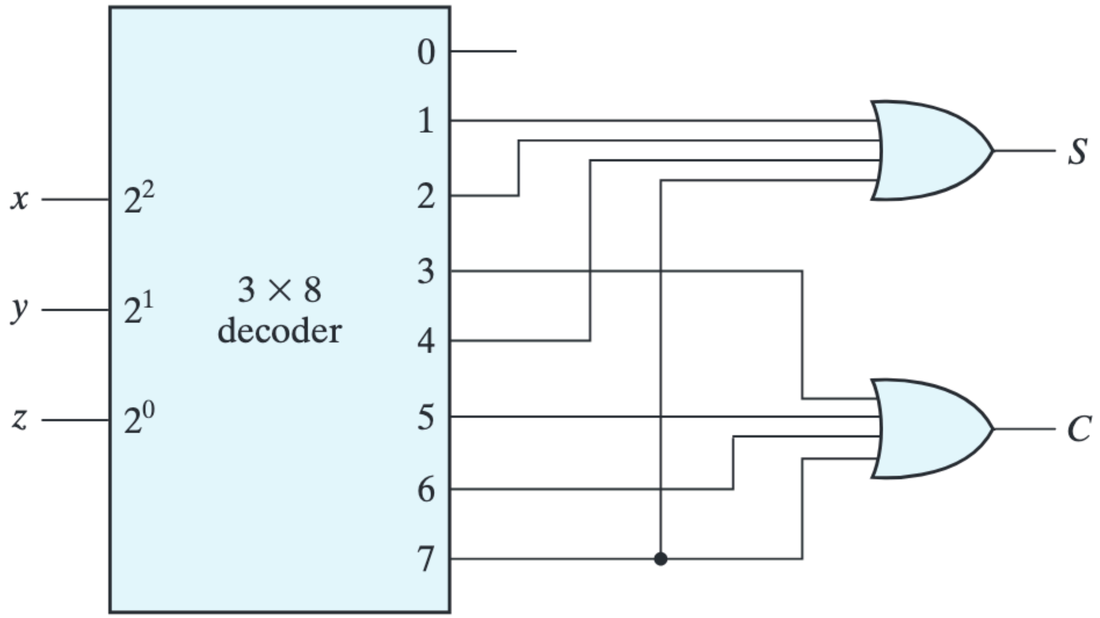
Three-to-Eight line decoder

- Binary-to-octal representation
  - Input binary number, say 110, asserts one output line D_6
- Each output represents a minterm



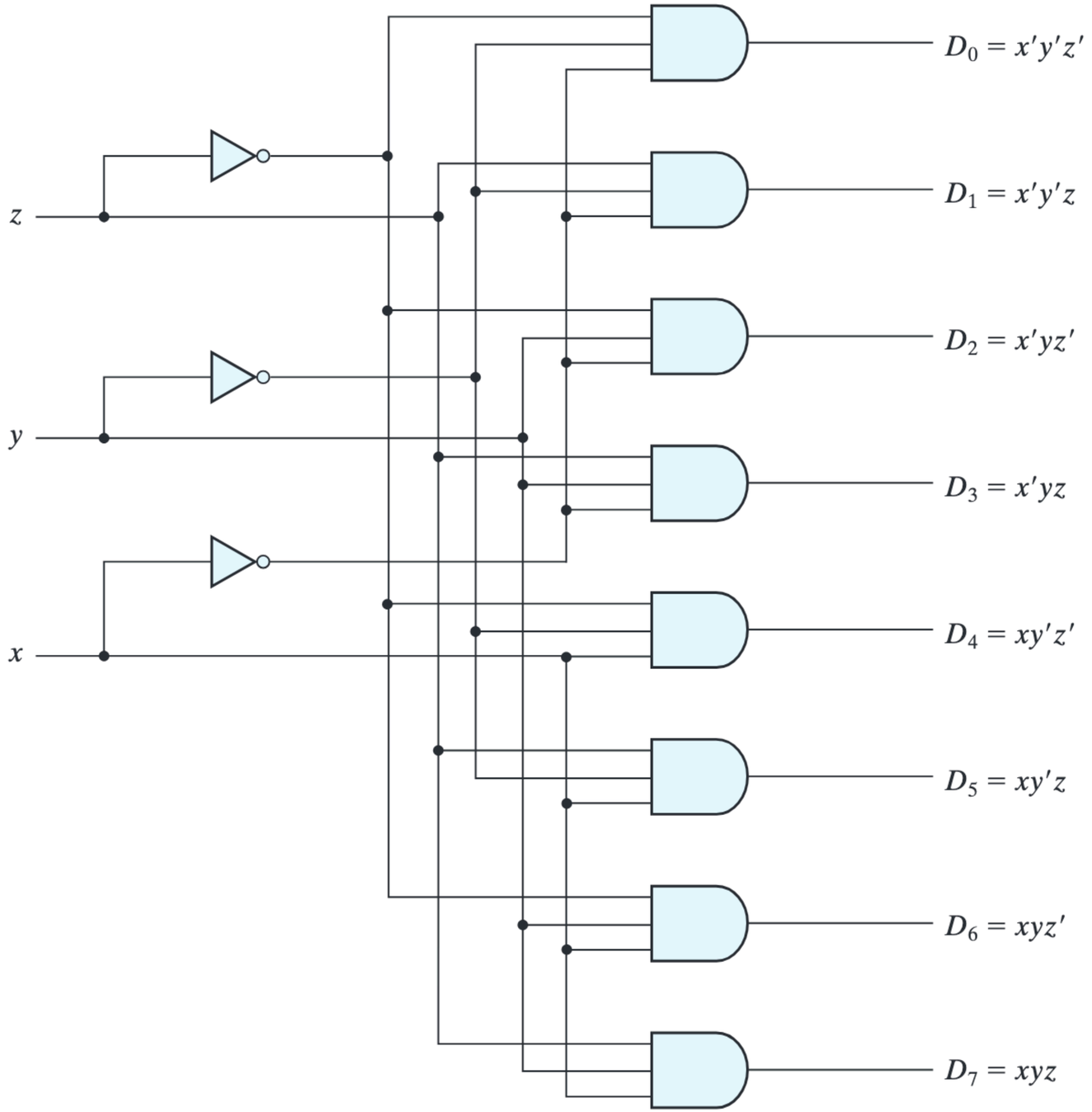
Generating Boolean Functions using Decoders

- Each Boolean Function can be expressed as sum-of-minterms
- Take an OR of all the minterms of the function
- Full adder
  - $S(x, y, z) = \Sigma(1,2,4,7)$
  - $C(x, y, z) = \Sigma(3,5,6,7)$



Generating Boolean Functions using Decoders

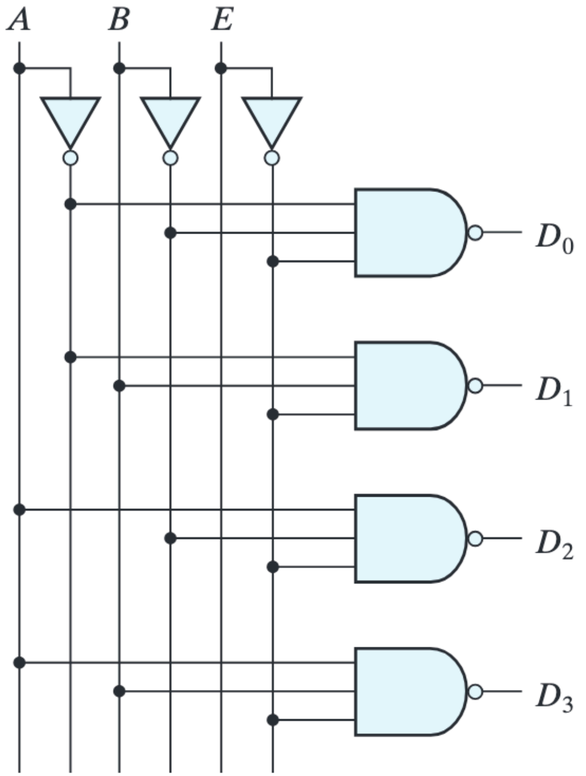
- If function F has $k > 2^n/2$ minterms, we can generate F' with $2^n - k$ minterms and use a NOR instead of an OR to get F
- This gives us an AND-NOR circuit. We can also create a NAND-AND circuit to generate the same function



NAND Decoder

E	A	B	D_0	D_1	D_2	D_3
1	X	X	1	1	1	1
0	0	0	0	1	1	1
0	0	1	1	0	1	1
0	1	0	1	1	0	1
0	1	1	1	1	1	0

- Since NAND gates are more economical: generate minterms in their complemented form, i.e., only one output is zero at a time
- Another input to enable/disable the decoder's outputs
  - Complemented outputs and complemented Enable (circuit outputs only when E=0)

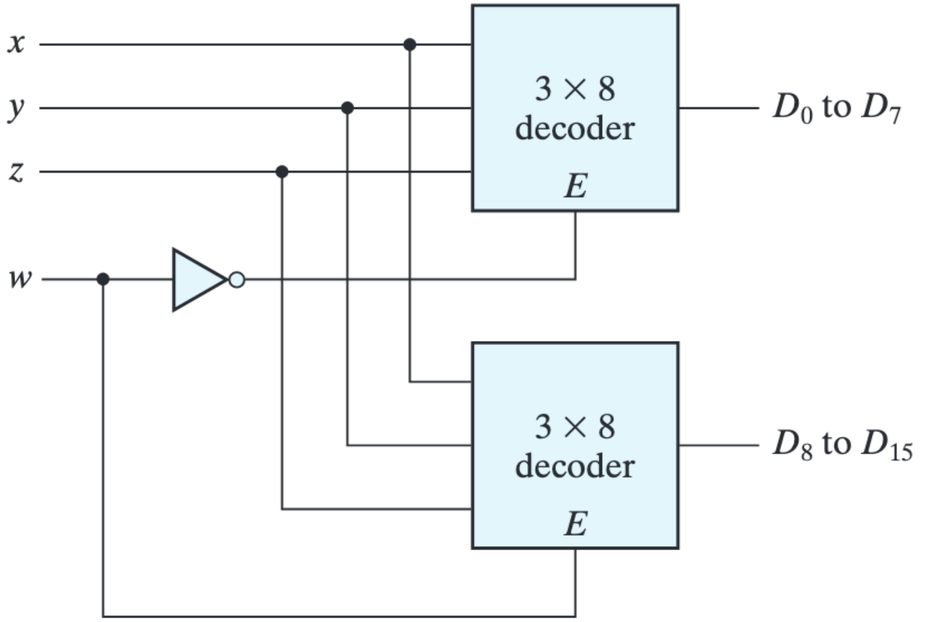


Decoder Demultiplexer

- Decoder is also called a *demultiplexer.* N-input lines enable a particular downstream circuit
  - CPU decodes an n-bit instruction and *enables* a particular downstream circuit
  - x86 32 bit.
    addl %eax, %ebx is 01 D8
    subl %eax, %ebx is 29 C3

Decoder Demultiplexer (2)

- w bit might come from an earlier decoder which enables appropriate downstream decoder
- What is this circuit?
- Enable inputs are a convenient feature for interconnecting two or more standard components



Encoders

- Inverse of decoder: converts 2^n (or fewer) input lines to n output lines
  - When a keyboard key is pressed, convert it to an ASCII code.
- It is assumed that only 1 input line is active at-a-time
- Can be implemented with OR gates

$$z = D_1 + D_3 + D_5 + D_7$$

$$y = D_2 + D_3 + D_6 + D_7$$

$$x = D_4 + D_5 + D_6 + D_7$$

Inputs								Outputs		
D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7	x	y	z
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1

Encoders

- What if multiple inputs (say D_3 and D_6) are simultaneously 1?
  - Outputs: 111. It represents D_7 instead of D_3 or D_6
- What if all inputs are zero?
  - Outputs: 000. It represents D_0

$$z = D_1 + D_3 + D_5 + D_7$$

$$y = D_2 + D_3 + D_6 + D_7$$

$$x = D_4 + D_5 + D_6 + D_7$$

Inputs								Outputs		
D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7	x	y	z
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1

Priority Encoders

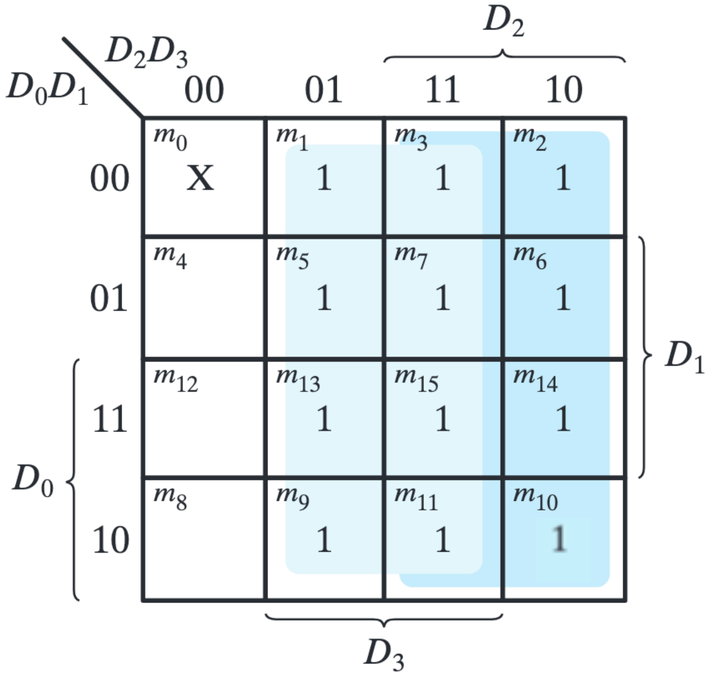
- What if multiple inputs (say D_3 and D_6) are simultaneously 1?
  - Decide a priority of inputs. Say higher subscript number trumps output. Output 110 when D_3 and D_6 are simultaneously 1.
- What if all inputs are zero?
  - Output a valid bit. When all inputs are zero, valid bit is zero

Inputs				Outputs		
D_0	D_1	D_2	D_3	x	y	V
0	0	0	0	X	X	0
1	0	0	0	0	0	1
X	1	0	0	0	1	1
X	X	1	0	1	0	1
X	X	X	1	1	1	1

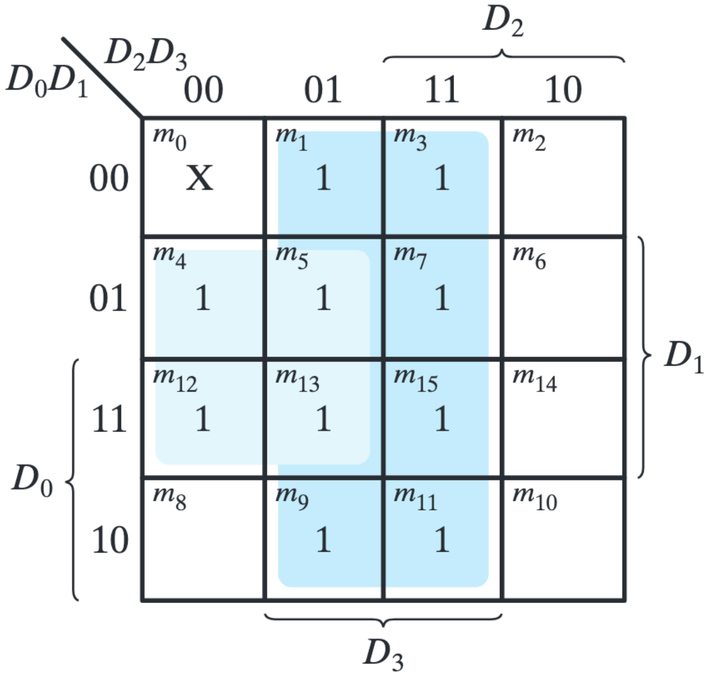
Priority Encoder (2)

- Higher subscript number trumps output
- When all inputs are zero, valid bit is zero.
- $V = D_0 + D_1 + D_2 + D_3$

Inputs				Outputs		
D_0	D_1	D_2	D_3	x	y	V
0	0	0	0	X	X	0
1	0	0	0	0	0	1
X	1	0	0	0	1	1
X	X	1	0	1	0	1
X	X	X	1	1	1	1



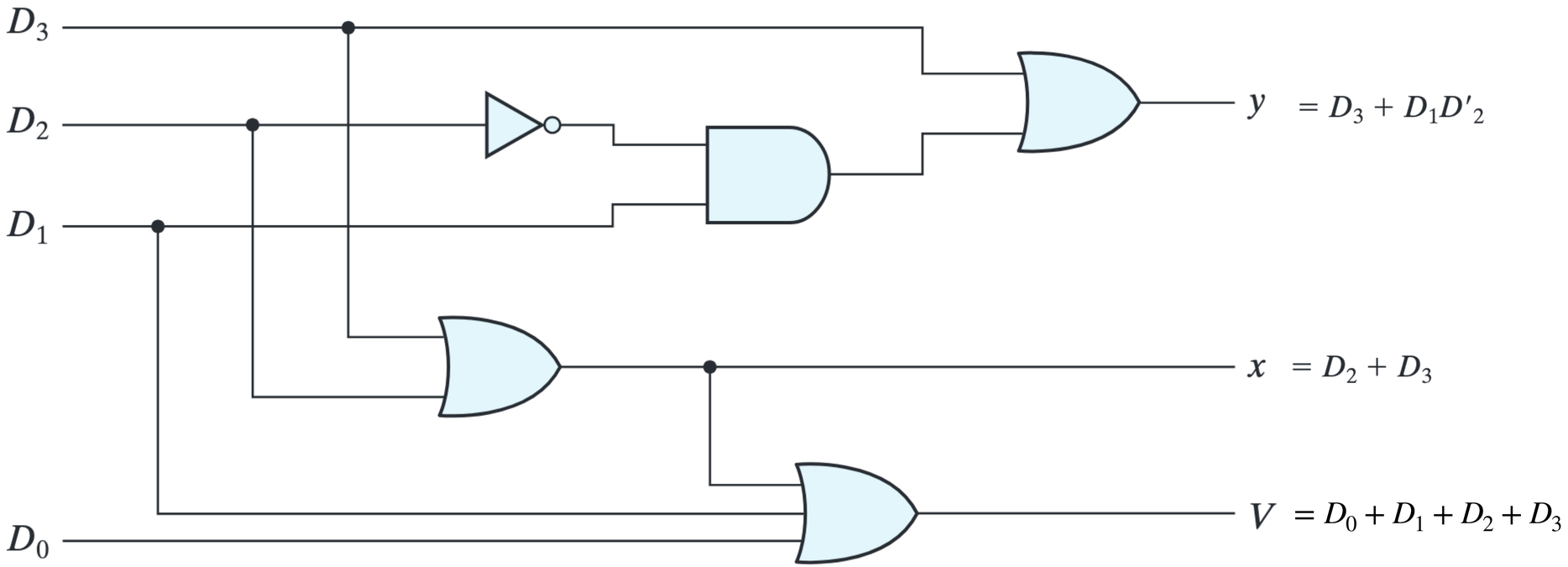
$x = D_2 + D_3$



$y = D_3 + D_1D'_2$

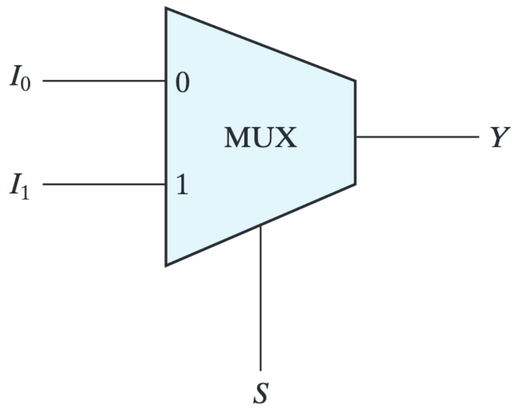
Priority Encoder (3)

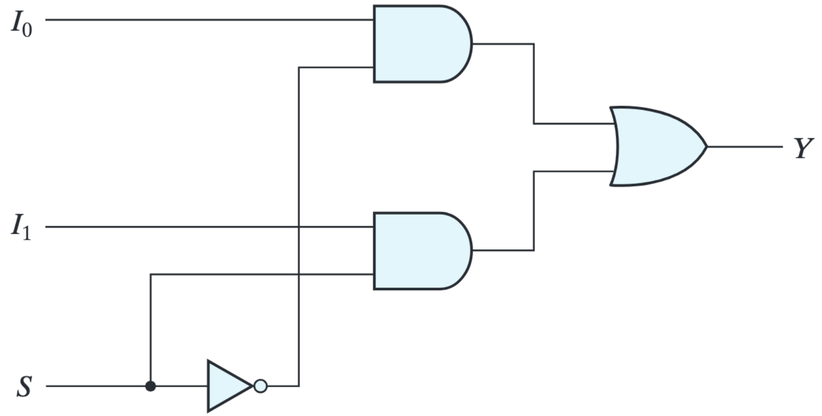
- Higher subscript number trumps output
- When all inputs are zero, valid bit is zero



Multiplexers

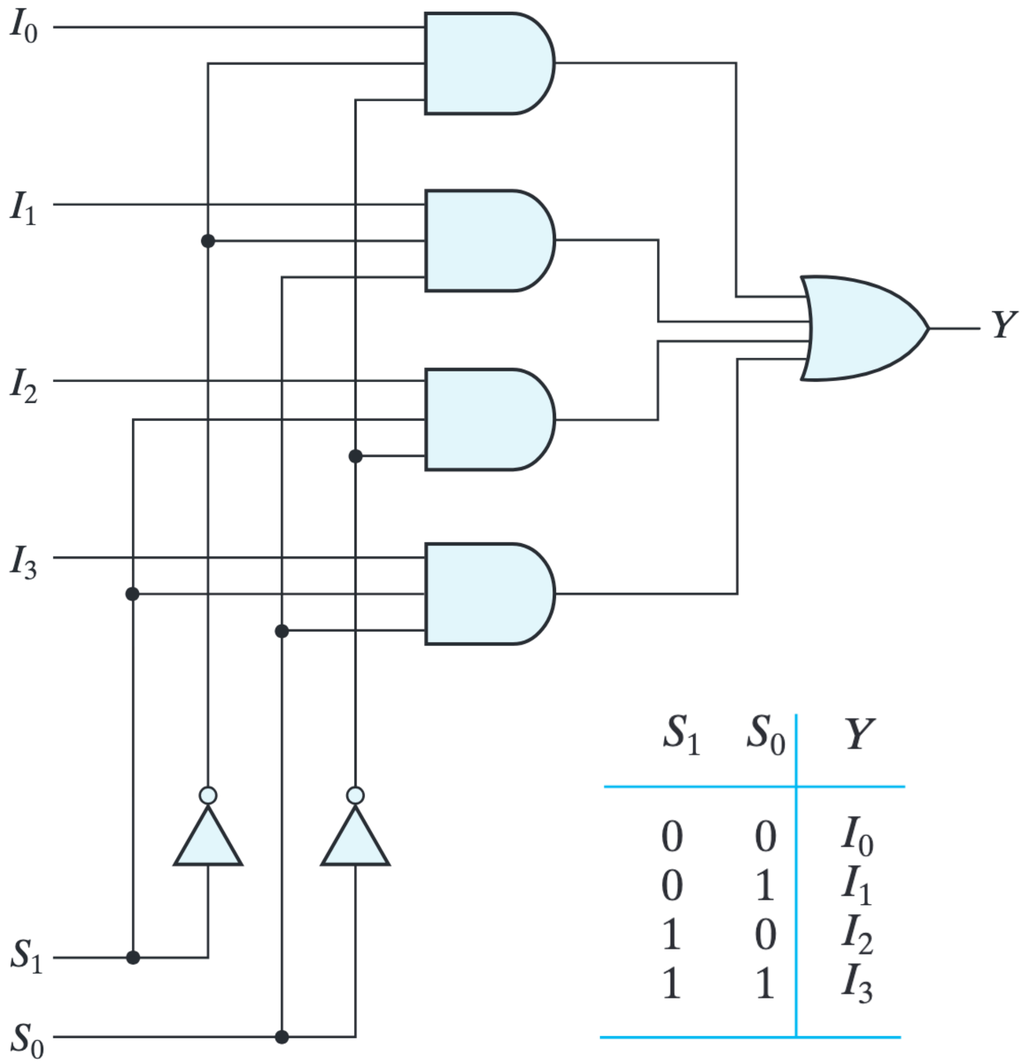
- Select one of many input lines and direct to a single output line
- *Selection lines* control which input is selected
- Multiplexer is also called a “data selector”





Four-to-one line multiplexer

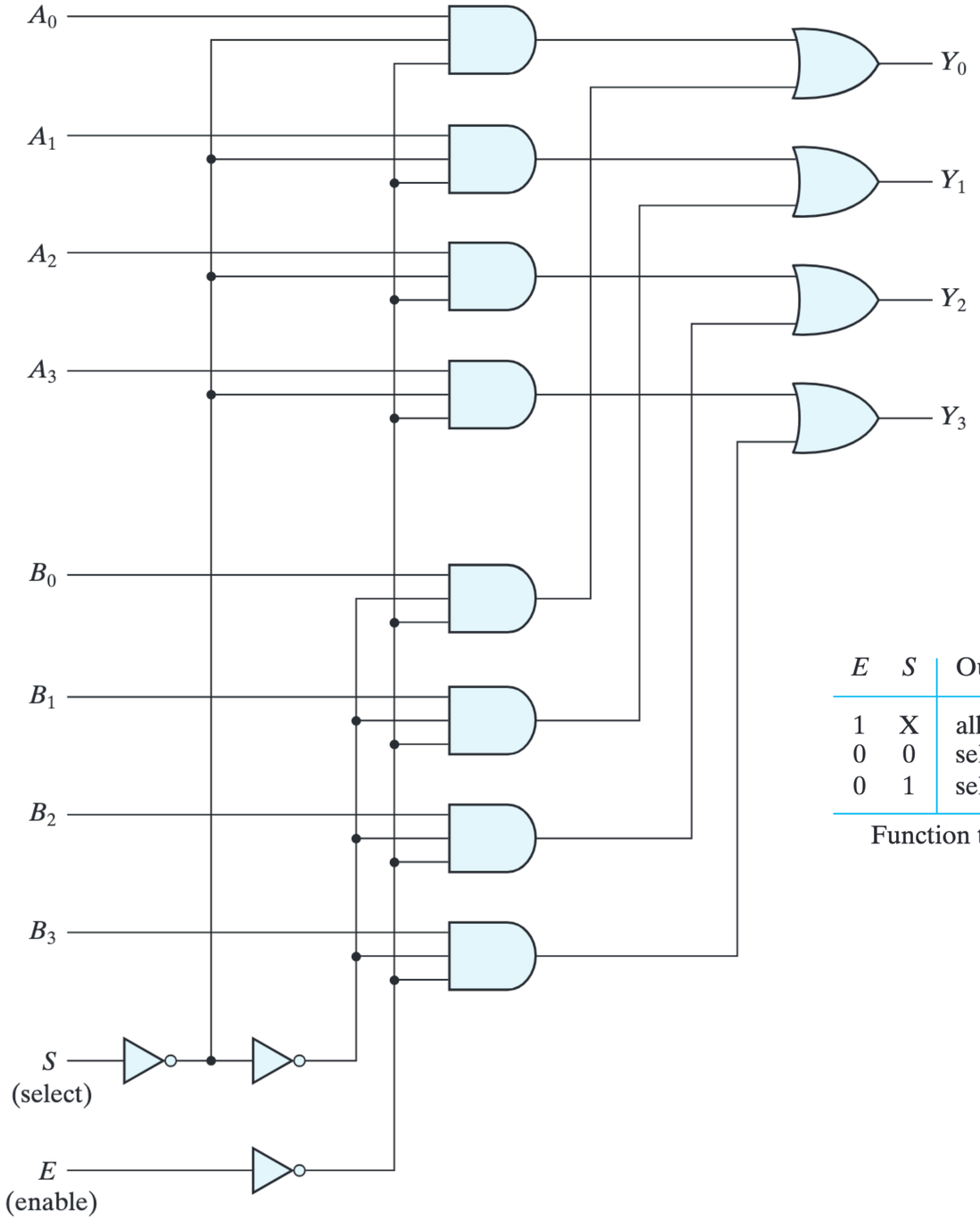
- "Function table" identifies relationship of output with input
- Resembles a decoder from selection lines to output lines. For a 2^n to 1 mux,
  - Use a n to 2^n decoder
  - Apply inputs to the AND gates
  - Follow the AND gates with an OR gate



S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

Multi-bit selection

- 4-bit inputs $A = A_3A_2A_1A_0$ and $B = B_3B_2B_1B_0$
- 4-bit outputs $Y = Y_3Y_2Y_1Y_0$
- When enable bit is 1, mux is inactive, i.e., output is 0

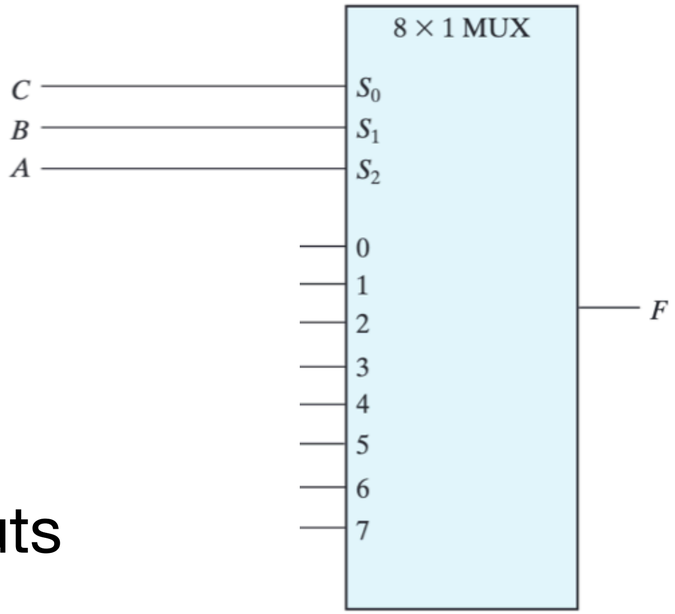


E	S	Output Y
1	X	all 0's
0	0	select A
0	1	select B

Function table

Implementing Boolean Function with Multiplexer

- $F(A, B, C) = \Sigma(1,2,4,7)$
  - Connect A, B, C to selection inputs
  - $I_1, I_2, I_4, I_7 = 1.\ I_0, I_3, I_5, I_6 = 0$
- Implementing n variable Function with **n** selection inputs
  - Connect n variables to selection inputs
  - Select 0 or 1 as per min-terms in the truth table



Implementing Boolean Function with Multiplexer

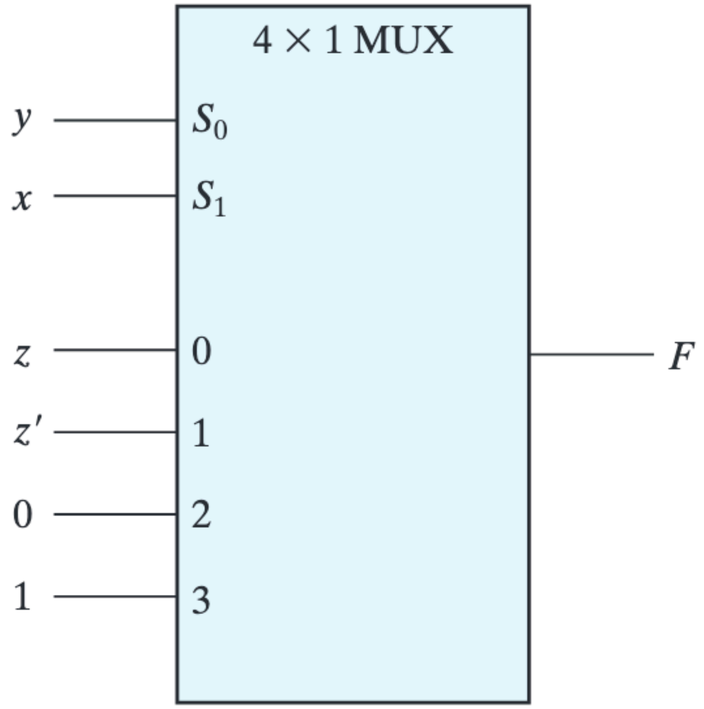
- Implementing n variable Function with **n-1** selection inputs
- Connect first n-1 variables to selection inputs
- Use remaining single variable (say z) in the data input
  - Each data input can be either 0, 1, z, or z’

Implementing Boolean Function with Multiplexer

- $F(x, y, z) = \Sigma(1,2,6,7)$
- When xy=00, data path 0 will get selected
- When xy=00, F=z. So apply z at the data path 0
- Note that F can only be equal to 0, 1, z, or z’

x	y	z	F	
0	0	0	0	$F = z$
0	0	1	1	
0	1	0	1	$F = z'$
0	1	1	0	
1	0	0	0	$F = 0$
1	0	1	0	
1	1	0	1	$F = 1$
1	1	1	1	

(a) Truth table



(b) Multiplexer implementation

Implementing Boolean Function with Multiplexer

$$F(A,B,C,D) =$$

$$\Sigma(1,3,4,11,12,13,14,15)$$

Exercise: Implement

$$F(A,B,C) = \Sigma(3,5,6,7)$$

A	B	C	D	F	
0	0	0	0	0	$F = D$
0	0	0	1	1	
0	0	1	0	0	$F = D$
0	0	1	1	1	
0	1	0	0	1	$F = D'$
0	1	0	1	0	
0	1	1	0	0	$F = 0$
0	1	1	1	0	
1	0	0	0	0	$F = 0$
1	0	0	1	0	
1	0	1	0	0	$F = D$
1	0	1	1	1	
1	1	0	0	1	$F = 1$
1	1	0	1	1	
1	1	1	0	1	$F = 1$
1	1	1	1	1	

